# Tema 4

## Razonamiento Basado en Reglas





# Indice

- Introducción
- Arquitectura de un SBR
  - Base de hechos y base de reglas
  - Motor de inferencia
- CLIPS
  - Base de hechos en CLIPS
  - Base de reglas en CLIPS
  - Pattern-matching
  - Unificación de reglas
- Motor de inferencia
- Estrategias de resolución de conflictos





# Ejemplo: edificio inteligente

Imaginemos un edificio inteligente con sensores que miden la temperatura de las habitaciones.
Queremos que el sistema:

- Reconozca cuándo hace calor.
- Y si hace calor, que encienda el aire acondicionado.

¿Podemos resolver este problema con OWL y SWRL?





# Ejemplo: edificio inteligente

## Ontología base (OWL)

- Sensor
- Habitación
- HabitacionTempAlta
- AireAcondicionado
- mide(Sensor,Habitación)
- tieneTemperatura(Sensor, valor)
- tieneAA(Habitación, AireAcondicionado)
- activoAA(AireAcondicionado)





# Ejemplo: edificio inteligente

- Supongamos que en la ontología tenemos estas dos reglas:

Regla 1: Sensor(?s) ∧ mide(?s, ?h) ∧ tieneTemperatura(?s, ?t) ∧ greaterThan(?t, 30) → HabitacionTempAlta(?h)

Regla 2: HabitacionTempAlta(?h) ∧ tieneAA(?h, ?a) → activoAA(?a, true)

- Las reglas SWRL expresan consecuencias lógicas, no una orden de actuación externa
- La inferencia de activoAA() no enciende físicamente el aparato





# Ejemplo: edificio inteligente

- Con un SBR es posible:
  - Indicar qué acción a realizar
  - Cambiar el estado del sistema

**El sistema puede reaccionar automáticamente a nuevos hechos**



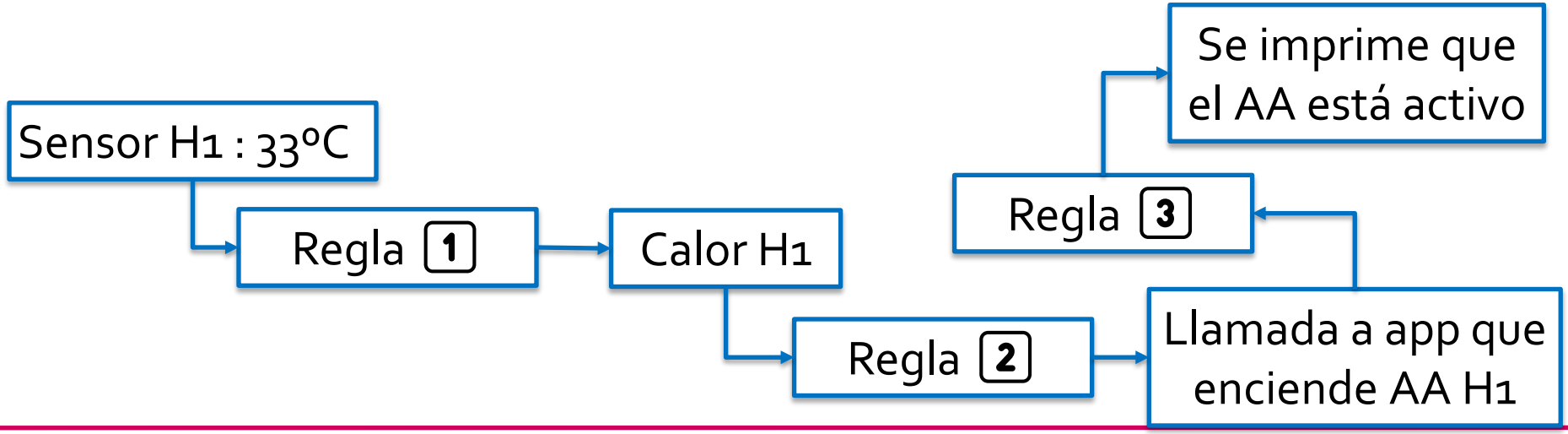


# Ejemplo: edificio inteligente

- Si un SBR se definen las siguientes reglas:

1. Si la temperatura de una habitación supera 30°C → se detecta calor.
2. Si hay calor en una habitación → se enciende el aire acondicionado.
3. Si el aire está encendido → se confirma que la habitación está climatizada.

  - Cada nueva conclusión activa la siguiente regla.
  - El sistema cambia de estado y actúa.








# Ejemplo: edificio inteligente



## Ontologías

- Describen *qué cosas existen y cómo se relacionan*
- Si se utilizan reglas SWRL, se puede indicar *qué debe deducirse cuando algo ocurre*
- **OWL y SWRL** representan conocimiento y permiten derivar consecuencias lógicas.

## Sistemas de producción

- Puede ejecutar acciones (assert, retract, salidas o llamadas externas) que modifican su memoria de trabajo.
- Su motor controla qué activaciones se ejecutan y en qué orden





# Definición

Un **sistema basado en reglas** es un sistema basado en el conocimiento que representa el conocimiento mediante un conjunto de *reglas de producción* del tipo:

"SI condición ENTONCES acción o conclusión"

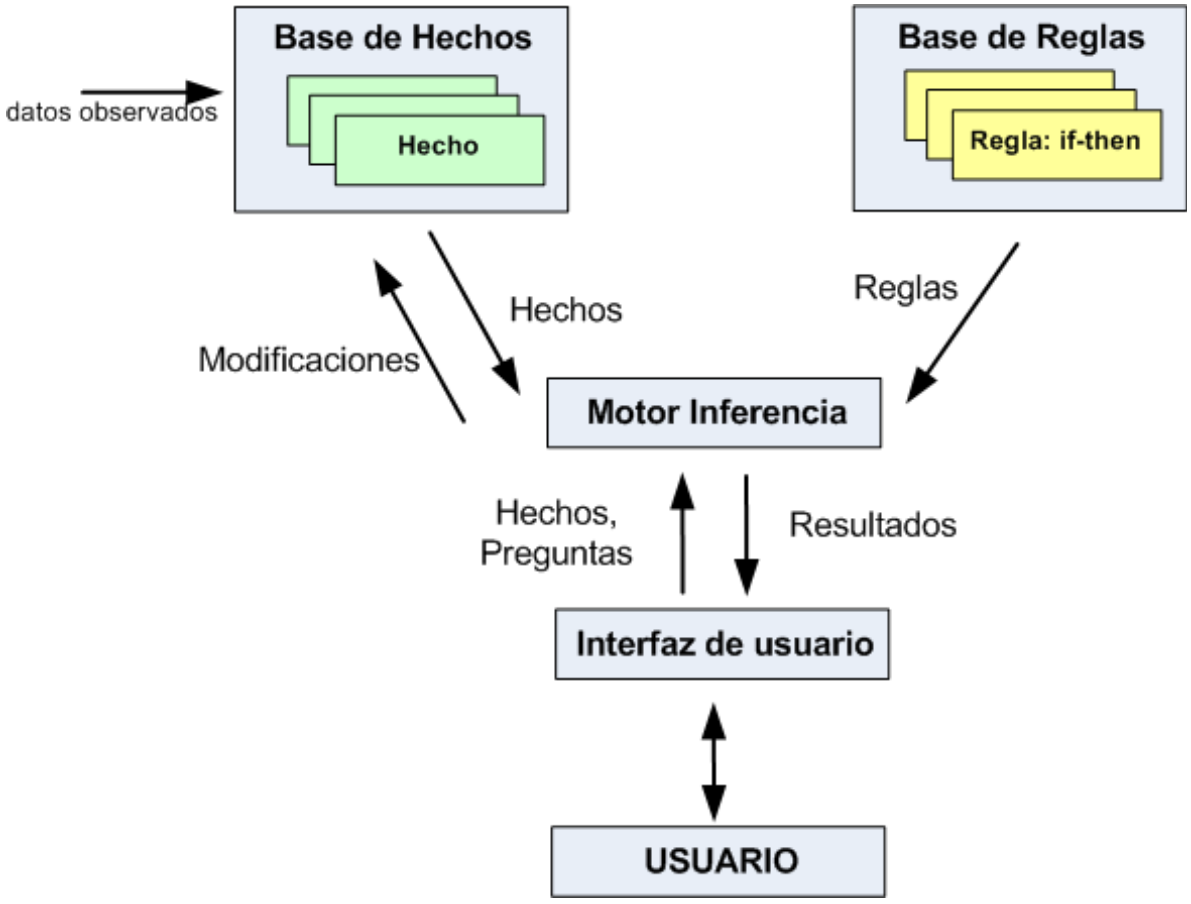
y utiliza un *motor de inferencia* para aplicar dichas reglas sobre una *base de hechos* con el fin de obtener nuevas conclusiones o tomar decisiones.





# Sistemas basados en reglas (SBR)

También llamados Sistemas de Producción, son sistemas que utilizan reglas como la base de la representación del conocimiento








## Base de Reglas

- Contiene ***reglas***, cada regla representa un paso de la resolución del problema. Una regla es un conjunto de condiciones (precondiciones) junto con las operaciones a realizar si las condiciones se satisfacen (efectos).
- Las reglas son conocimiento duradero (persistente) sobre el dominio
- Típicamente, una vez se han definido las reglas para un problema, éstas no se modifican, solo el experto del dominio puede modificarlas

Reglas del tipo if-then:

- $p1 \wedge p2 \wedge \ldots \wedge pn \Rightarrow a1, a2, \ldots, an$
- Antecedente (premisa o condición): conjunto de condiciones (sentencias en lógica de predicados, condiciones sobre objetos ...)
  - Si un hecho no está en la BH, se considera falso (mundo cerrado)
- Consecuente (conclusión o acción): conjunto de operaciones. Una operación puede ser:
  - Añadir un hecho a la BH
  - Eliminar un hecho de la BH
  - Consultas al usuario
  - Acciones externas (print, read, ....)





# Base de hechos y base de reglas

## Base de Hechos

- También llamada memoria de trabajo o base de datos
- Contiene ***hechos*** sobre el mundo, que pueden observarse directamente o deducirse a través de las reglas
- Contiene conocimiento temporal, conocimiento sobre uno o varios estados de la resolución del problema
- Se puede modificar mediante reglas (añadir hechos, eliminar hechos)





# Ejemplo

- Un sistema supervisa los servidores de un centro de datos. Queremos determinar si un servidor necesita atención urgente.
- **Base de reglas:**

  R1: temperaturaAlta(x) ⇒ add(sobrecalentamiento(x))

  R2: sobrecalentamiento(x) ∧ ventiladorAveriado(x) ⇒ add(riesgoCritico(x))

  R3: riesgoCritico(x) ⇒ add(requiereIntervencion(x)), sonarAlarma(x)

- **Base de hechos**:

  temperaturaAlta(Servidor1)

  ventiladorAveriado(Servidor1)

- ¿Qué hechos podemos deducir y qué acciones se ejecutarían?





# Ejercicio

**Sistema basado en reglas para la seguridad de edificios**

Se desea desarrollar un sistema basado en reglas que permita detectar posibles intrusiones en los edificios de un campus universitario y ejecutar las acciones de seguridad correspondientes. El sistema recibe información de distintos sensores instalados en los edificios y debe actuar de acuerdo con las siguientes condiciones:

- Si se detecta movimiento en un edificio durante el horario nocturno, el sistema considera que existe actividad sospechosa.
- Si existe actividad sospechosa en un edificio y alguna de sus puertas está abierta, el sistema considera que se ha producido una posible intrusión.
- Cuando se detecta una posible intrusión, el sistema debe registrar una alerta de seguridad y ejecutar la acción de activar la alarma del edificio.
- Cuando existe una alerta de seguridad en un edificio, el sistema debe ejecutar la acción de notificar al vigilante correspondiente.

Inicialmente, el sistema dispone de la siguiente información:

- Edificio A: se ha detectado movimiento, es horario nocturno y una puerta está abierta.
- Edificio B: se ha detectado movimiento y es horario nocturno. No se dispone de información sobre el estado de sus puertas.
- Edificio C: se ha detectado movimiento, es horario diurno y una puerta está abierta.





# Ejercicio

**Sistema basado en reglas para la seguridad de edificios**

Se pide:

a) Diseña la base de hechos inicial, utilizando predicados adecuados para representar la información disponible.

b) Diseña la base de reglas, utilizando reglas de producción de la forma condición → acción. Distingue entre las acciones que incorporan nuevos hechos a la base de hechos y las que ejecutan operaciones externas.





# Motor de inferencia

- Procesa la información de la BH y BR
- Es el mecanismo de razonamiento, independiente del dominio, para SBR
- Selecciona una regla de la BR y la aplica
- Un motor de inferencia se caracteriza por:
  - El tipo de encadenamiento
  - El proceso de unificación (pattern-matching)
  - El mecanismo de control para seleccionar y ejecutar reglas
- Dos tipos de motor de inferencia:
  - Encadenamiento hacia delante o razonamiento dirigido por los datos (las reglas unifican el antecedente e infieren el consecuente)
  - Encadenamiento hacia atrás o razonamiento dirigido por el objetivo (las reglas unifican el consecuente y prueban el antecedente)





# Encadenamiento hacia delante

- Si las precondiciones de una regla se satisfacen, se infiere la conclusión
- Un motor de inferencia con encadenamiento hacia delante se puede describir lógicamente como una aplicación repetida del *modus ponens*.
- Uso típico: si queremos inferir todas las sentencias que se deducen de una base de conocimiento





# Ejemplo

- Base de conocimiento

R1: temperaturaAlta(x) ⇒ add(sobrecalentamiento(x))
R2: sobrecalentamiento(x) ∧ ventiladorAveriado(x) ⇒ add(riesgoCritico(x))
R3: riesgoCritico(x) ⇒ add(requiereIntervencion(x)), sonarAlarma(x)

F1: temperaturaAlta(Servidor1)
F2: ventiladorAveriado(Servidor1)

¿Qué hechos podemos deducir y qué acciones se ejecutarían?

- Procedimiento

Se satisface R1: ⇒ add(sobrecalentamiento(Servidor1)) → F3
Se satisface R2: ⇒ add(riesgoCritico(Servidor1)) → F4
Se satisface R3: ⇒ add(requiereIntervencion(Servidor1)) → F5
sonarAlarma(Servidor1)

BH:
F1: temperaturaAlta(Servidor1)
F2: ventiladorAveriado(Servidor1)
F3: sobrecalentamiento(Servidor1)
F4: riesgoCritico(Servidor1)
F5: requiereIntervención(Servidor1)





# Ejercicio

**Sistema basado en reglas para la seguridad de edificios**

Dada la base de hechos inicial y la base de reglas del ejercicio anterior, se pide:

a) Determina qué reglas pueden aplicarse inicialmente y qué nuevos hechos se obtienen al aplicar sucesivamente las reglas hasta que no puedan derivarse más hechos.

b) Indica la base de hechos final y las acciones externas que se habrán ejecutado, especificando a qué edificios afectan.





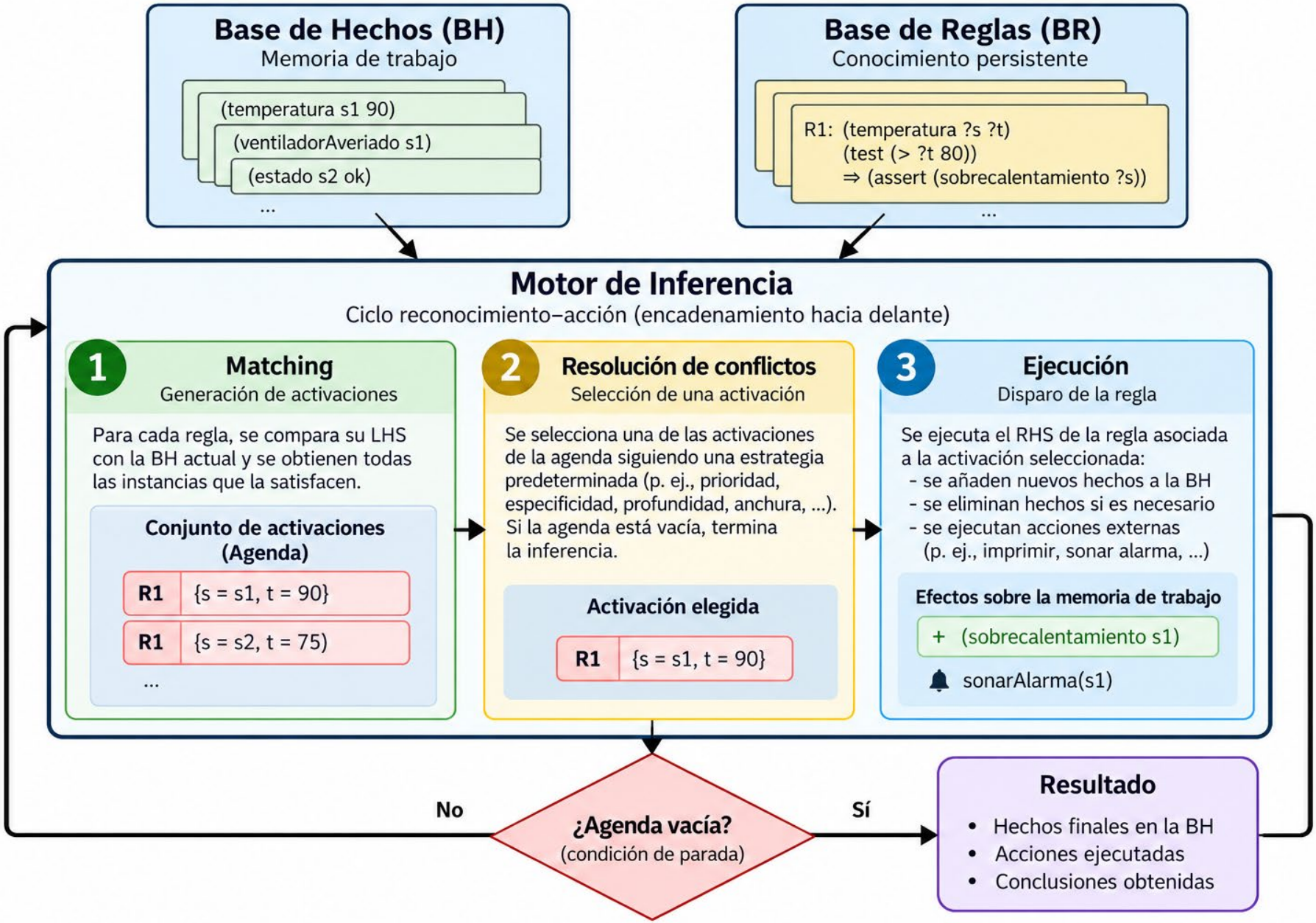
# Motor de inferencia

Motor de inferencia hacia delante (tres operaciones):

1. **Pattern-matching**: unificación del antecedente de las reglas de la Base de Reglas (BR) con los contenidos de la Base de Hechos (BH). El resultado es un conjunto de reglas instanciadas o aplicables (instancias de reglas).
2. **Selección de una instancia de regla**
3. **Ejecución del consecuente de la instancia de regla** y actualización de la BH






Base de Hechos (BH)
Memoria de trabajo
(temperatura s1 90)
(ventiladorAveriado s1)
(estado s2 ok)
...
Base de Reglas (BR)
Conocimiento persistente
R1: (temperatura ?s ?t)
(test (> ?t 80))
⇒ (assert (sobrecalentamiento ?s))
...
Motor de Inferencia
Ciclo reconocimiento–acción (encadenamiento hacia delante)
1
Matching
Generación de activaciones
Para cada regla, se compara su LHS con la BH actual y se obtienen todas las instancias que la satisfacen.
Conjunto de activaciones (Agenda)
R1 {s = s1, t = 90}
R1 {s = s2, t = 75)
...
2
Resolución de conflictos
Selección de una activación
Se selecciona una de las activaciones de la agenda siguiendo una estrategia predeterminada (p. ej., prioridad, especificidad, profundidad, anchura, ...). Si la agenda está vacía, termina la inferencia.
Activación elegida
R1 {s = s1, t = 90}
3
Ejecución
Disparo de la regla
Se ejecuta el RHS de la regla asociada a la activación seleccionada:
- se añaden nuevos hechos a la BH
- se eliminan hechos si es necesario
- se ejecutan acciones externas (p. ej., imprimir, sonar alarma, ...)
Efectos sobre la memoria de trabajo
+ (sobrecalentamiento s1)
sonarAlarma(s1)
¿Agenda vacía?
(condición de parada)
No
Sí
Resultado
Hechos finales en la BH
Acciones ejecutadas
Conclusiones obtenidas

# Dos iteraciones del motor de inferencia aplicado al problema de sensores en el centro de datos

F1: temperaturaAlta(Servidor1)
F2: ventiladorAveriado(Servidor1)
F3: temperaturaAlta(Servidor2)

Matching

R1: temperaturaAlta(x) ⇒ add(sobrecalentamiento(x))
R2: sobrecalentamiento(x) ∧ ventiladorAveriado(x) ⇒ add(riesgoCritico(x))
R3: riesgoCritico(x) ⇒ add(requiereIntervencion(x)), sonarAlarma(x)

Agenda

R1 (F1): x=Servidor1
R1 (F3): x=Servidor2

Resolución de conflictos

R1

Ejecución

F1: temperaturaAlta(Servidor1)
F2: ventiladorAveriado(Servidor1)
F3: temperaturaAlta(Servidor2)
F4: sobrecalentamiento(Servidor1)





# Dos iteraciones del motor de inferencia aplicado al problema de sensores en el centro de datos

F1: temperaturaAlta(Servidor1)
F2: ventiladorAveriado(Servidor1)
F3: temperaturaAlta(Servidor2)
F4: sobrecalentamiento(Servidor1)

Matching

R1: temperaturaAlta(x) ⇒ add(sobrecalentamiento(x))
R2: sobrecalentamiento(x) ∧ ventiladorAveriado(x) ⇒ add(riesgoCritico(x))
R3: riesgoCritico(x) ⇒ add(requiereIntervencion(x)), sonarAlarma(x)

Agenda

R1 (F3): x=Servidor2
R2 (F4,F2): x=Servidor1

Resolución de conflictos

R2

Ejecución

F1: temperaturaAlta(Servidor1)
F2: ventiladorAveriado(Servidor1)
F3: temperaturaAlta(Servidor2)
F4: sobrecalentamiento(Servidor1)
F5: riesgoCritico(Servidor1)





# Ejercicio

## Sistema basado en reglas para la seguridad de edificios

Dada la base de hechos inicial y la base de reglas del ejercicio anterior, realiza 5 iteraciones del motor de inferencia indicando en cada caso las activaciones que aparecen en la agenda, la que se selecciona para su ejecución y el estado de la base de hechos tras cada iteración.





# CLIPS: una herramienta para la construcción de SBR

CLIPS (C Language Implementation Production System) es una herramienta que proporciona todas las funcionalidades para la construcción de SBR. CLIPS ha sido desarrollada por el Software Technology Branch (STB), NASA/Lyndon B. Johnson Space Center.

CLIPS proporciona:

- Mecanismos para definir los *hechos* de la BH
- Mecanismos para definir las *reglas* de la BR
- También proporciona soporte para representar conocimiento procedural y orientado a objetos
- Un motor de inferencia hacia delante basado en el algoritmo de match Rete
- Diferentes estrategias para la resolución de conflictos (selección de una instancia de regla)





# Base de Hechos en CLIPS

Los hechos son los elementos de información básica. La BH es una lista de hechos.

Un hecho es una lista de valores atómicos que se referencian posicionalmente (hecho ordenado)

***Hecho ordenado***: símbolo seguido de cero o más elementos separados por espacios en blanco y delimitados por un paréntesis de apertura a la izquierda y un paréntesis de cierre a la derecha. El primer elemento de un hecho es un símbolo que establece una 'relación' con el resto de elementos del hecho ordenado.

(símbolo <elemento>*)  elemento: símbolo, int, float, string

Ejemplos de hechos:

(temperaturaAlta Servidor1)
(temperaturaAlta Servidor2)
(location kitchen)
(on blockA blockB)
(tower blockA blockB blockC)

(ventiladorAveriado Servidor1)
(sobrecalentamiento Servidor1)
(holds Mary apple)
(on blockB table)
(person name "John" age 24 activity journalist)





# Base de Hechos en CLIPS

Constructor para definir un grupo inicial de hechos:

**(deffacts <deffacts-name> [<comment>]  <fact>*)**

La BH contiene una lista de hechos que describen una situación concreta del mundo (estado del mundo).

Mundo cerrado: la ausencia de un hecho indica que la información es falsa.

Representación de estados de un problema mediante hechos:

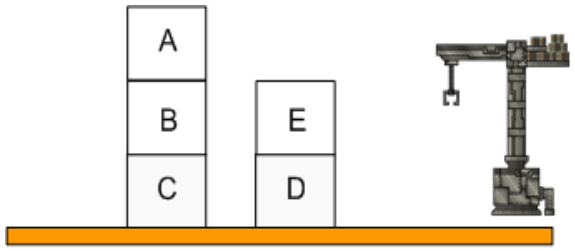
```
(deffacts servers
     (temperaturaAlta Servidor1)
     (ventiladorAveriado Servidor1)
     (temperaturaAlta Servidor2)
)
```

```
BH={ f-1: (temperaturaAlta Servidor1)
       f-2: (ventiladorAveriado Servidor1)
       f-3: (temperaturaAlta Servidor2)
}
```





# Base de Hechos en CLIPS

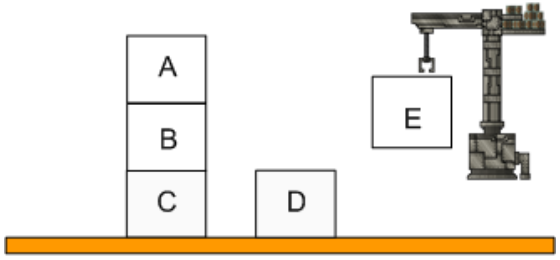



initial state (s0)

Asumiendo que la altura máxima de las torres es de tres bloques y que la mesa es ilimitada

```
(deffacts blocks
    (tower maximum height 3)
    (blocks tower 1 A B C tower 2 E D hoist nothing))
```

BH={ f-1: (tower maximum height 3)
f-2: (blocks tower 1 A B C tower 2 E D hoist nothing)}




state s1

BH={ f-1: (tower maximum height 3)
f-3: (blocks tower 1 A B C tower 2 D hoist E)}





# Ejercicio

**Sistema basado en reglas para la seguridad de edificios**

Diseña una base de hechos inicial en CLIPS para este problema. Recuerda que la información de la que el sistema dispone inicialmente es:

- Edificio A: se ha detectado movimiento, es horario nocturno y una puerta está abierta.
- Edificio B: se ha detectado movimiento y es horario nocturno. No se dispone de información sobre el estado de sus puertas.
- Edificio C: se ha detectado movimiento, es horario diurno y una puerta está abierta.

Se pide:

a) Diseña la BH utilizando hechos independientes para cada dato de cada edificio.

b) Diseña la BH utilizando hechos independientes para cada edificio, es decir, combinando la información disponible sobre cada edificio en el mismo hecho.





# Base de reglas en CLIPS

La BR contiene reglas. Una regla tiene dos partes:

- El antecedente, premisa o condición (Left-Hand Side –LHS- de la regla): representa las condiciones que deben satisfacerse en el estado actual para que la regla sea aplicable
- El consecuente, conclusión o acción (Right-Hand Side –RHS- de la regla): representa las acciones que se ejecutarán en el estado actual cuando la regla sea aplicable y se seleccione para ser ejecutada (disparo de la regla)

```
(defrule <nombre regla>
   (condicion1)
   (condicion2)
=>
   (accion1)
   (accion2)
)
```





# Parte izquierda de las reglas (LHS)

Los elementos condicionales de la LHS de una regla son de dos tipos:

- Patrón (pattern): restricción que se utiliza para determinar los hechos que satisfacen la condición especificada en el patrón. Un patrón es equivalente a un predicado en lógica de primer orden. Los patrones de una regla unifican con los hechos.
- Test: evalúa una expresión definida en la LHS de una regla.

```
(defrule detectar-riesgoCritico
    (sobrecalentamiento ?x)
    (ventiladorAveriado ?x)
=>
  ....
```

```
(defrule detectar-calentamiento
    (temperatura ?x)
    (test (> ?x 50))
=>
  ....
```

```
(defrule generar-alerta
    (temperaturaAlta ?s)
    (not (alertaActiva ?s))
=>
    ....
```

Todos los patrones y tests de la LHS de la regla se deben satisfacer para que la regla sea aplicable

1. Un patrón se satisface si existe al menos un hecho en la BH que hace matching con el patrón
2. Un patrón **not** se satisface si no existe ningún hecho en la BH que coincida con el patrón incluido en **not**.
3. Un test se satisface si se evalúa a TRUE





# Hipótesis de mundo cerrado (CWA)

- En CLIPS, la hipótesis de mundo cerrado (Closed World Assumption, CWA) se refleja en el uso de la negación por ausencia de hechos.
  - Si un hecho no está presente en la Base de Hechos (BH), no puede utilizarse para satisfacer un patrón positivo.
  - El patrón not de la parte izquierda (LHS) de una regla se satisface cuando no existe ningún hecho que coincida con él.

```
(defrule generar-alerta
      (temperaturaAlta ?s)
      (not (alertaActiva ?s))
=>
      ....
```

```
(temperaturaAlta Servidor1)
(temperaturaAlta Servidor2)
(alertaActiva Servidor2)
```

| Instancia | ¿Se satisface la LHS? | Motivo |
|---|---|---|
| ?s = Servidor1 | Sí | Tiene temperatura alta y no consta una alerta activa. |
| ?s = Servidor2 | No | Ya existe (alertaActiva Servidor2). |





# Parte derecha de las reglas (RHS)

Dos comandos principales:

- assert: añade hechos en la BH
- retract: borra hechos de la BH

antes de la ejecución: **BH**=

f-1: (temperaturaAlta Servidor1)
f-2: (ventiladorAveriado Servidor1)
f-3: (temperaturaAlta Servidor2)

```
(defrule generar-alerta
      (temperaturaAlta ?s)
      (not (alertaActiva ?s))
=>
      (assert (sobrecalentamiento ?s))
      (assert (alertaActiva ?s)))
```

después de la ejecución: **BH**=

f-1: (temperaturaAlta Servidor1)
f-2: (ventiladorAveriado Servidor1)
f-3: (temperaturaAlta Servidor2)
f-4: (sobrecalentamiento Servidor1)
f-5: (alertaActiva Servidor1)





# Parte derecha de las reglas (RHS)

Dos comandos principales:

- assert: añade hechos en la BH
- retract: borra hechos de la BH

antes de la ejecución: **BH**=

f-1: (temperaturaAlta Servidor1)
f-2: (ventiladorAveriado Servidor1)
f-3: (temperaturaAlta Servidor2)
f-4: (sobrecalentamiento Servidor1)
f-5: (reparacionCompletada Servidor1)

```
(defrule registrar-reparacion
    ?f1 <- (reparacionCompletada ?s)
    (sobrecalentamiento ?s)
    ?f2 <- (ventiladorAveriado ?s)
=>
    (retract ?f1 ?f2)
    (assert (ventiladorOperativo ?s)))
```

?f1 y ?f2 son variables que almacenan referencias a los hechos que satisfacen los patrones (reparacionCompletada ?s) y (ventiladorAveriado ?s), respectivamente.

Cuando la regla se dispara, el comando 'retract' borra ambos hechos de la BH

después de la ejecución: **BH**=

f-1: (temperaturaAlta Servidor1)
~~f-2: (ventiladorAveriado Servidor1)~~
f-3: (temperaturaAlta Servidor2)
f-4: (sobrecalentamiento Servidor1)
~~f-5: (reparacionCompletada Servidor1)~~
f-6: (ventiladorOperativo Servidor1)





# Ejercicio

**Sistema basado en reglas para la seguridad de edificios**

Diseña las siguientes reglas en CLIPS utilizando la primera versión de la BH del ejercicio anterior (es decir, hechos independientes para cada dato de cada edificio). Recordemos que las reglas deben reflejar las siguientes condiciones:

- Si se detecta movimiento en un edificio durante el horario nocturno, el sistema considera que existe actividad sospechosa.
- Si existe actividad sospechosa en un edificio y alguna de sus puertas está abierta, el sistema considera que se ha producido una posible intrusión.
- Cuando se detecta una posible intrusión, el sistema debe registrar una alerta de seguridad y ejecutar la acción de activar la alarma del edificio.
- Cuando existe una alerta de seguridad en un edificio, el sistema debe ejecutar la acción de notificar al vigilante correspondiente.





# Ejercicio 2

**Sistema basado en reglas para la gestión de una biblioteca**

Se desea desarrollar un sistema basado en reglas que permita gestionar los préstamos de libros de una biblioteca universitaria, detectar incidencias y determinar si los usuarios pueden realizar nuevos préstamos. El sistema dispone de información sobre los usuarios, los libros y los préstamos realizados, y debe actuar de acuerdo con las siguientes condiciones:

- R1: Si un usuario tiene un retraso en la devolución de algún libro (representado mediante el hecho (prestamoVencido...), el sistema debe registrar una incidencia asociada a dicho usuario.
- R2: Si un usuario acumula dos incidencias correspondientes a libros diferentes, el sistema considera que su cuenta debe bloquearse.
- R3: Si un usuario tiene la cuenta bloqueada, el sistema debe generar una notificación que le informe de que no puede realizar nuevos préstamos (con printout).
- R4: Si un libro está disponible y el usuario que lo solicita no tiene la cuenta bloqueada, el sistema debe autorizar el préstamo.
- R5: Si se autoriza un préstamo, el sistema debe registrar que el libro ya no está disponible.





# Ejercicio 2

Información inicial:

- La biblioteca dispone de tres usuarios: Ana, Luis y Marta, y de cinco libros: L1, L2, L3, L4 y L5.
- Ana tiene dos préstamos vencidos, correspondientes a L1 y L2. Luis tiene un préstamo vencido de L3. Marta no tiene préstamos vencidos.
- Los libros L1, L2 y L3 están prestados, mientras que L4 y L5 están disponibles.
- Marta solicita el préstamo de L4 y Ana solicita el préstamo de L5.

Se pide:

- Diseña la base de hechos inicial (BH) utilizando hechos ordenados.
- Diseña la base de reglas (BR) en CLIPS para representar las condiciones descritas, utilizando variables que permitan aplicar las reglas a cualquier usuario y libro.
- Determina qué hechos nuevos pueden inferirse a partir de la BH inicial si las reglas se ejecutan en el orden R1->R2->R3->R4->R5.
- Explica qué ocurriría si se ejecutaran las reglas R4 y R5 antes de R1, R2 y R3.





# Pattern-matching (unificación de patrones)

Unificación de patrones (pattern matching o *unificación sintáctica*). Es el proceso de unificación de los patrones de la parte izquierda de una regla con los hechos de la BH. El motor de inferencia de CLIPS unifica automáticamente los patrones de las reglas de la BR con los hechos de la BH y determina las reglas que son aplicables (aquellas que satisfacen su LHS).

Los patrones pueden contener:

Constantes

Variables mono-valuadas: variable cuyo nombre comienza por un signo de interrogación '?'. Las variables mono-valuadas se ligan a un solo valor.

Variables multi-valuadas: variable cuyo nombre comienza con el signo '$?'. Las variables multi-valuadas se ligan a cero o más valores.

Comodín (wildcards). Su nombre es simplemente el símbolo '?' or '$?'. Son variables especiales que se pueden ligar a cualquier constante o conjunto de constantes pero sin mantener los valores ligados; el propósito de los comodines es unificar valores del hecho sin mantener los valores ligados a una variable (en caso de que esto no sea necesario).





# Pattern-matching (unificación de patrones)

## Unificación (matching)

| Elemento del patrón | Elemento del hecho |
|---|---|
| Constante | Constante |
| Variable mono-valuada (?x) | Un único valor |
| Variable multi-valuada ($?x) | Cero o más valores |
| Comodín mono-valuado (?) | Un único valor sin mantener la ligadura |
| Comodín multi-valuado ($?) | Cero o más valores sin mantener la ligadura |

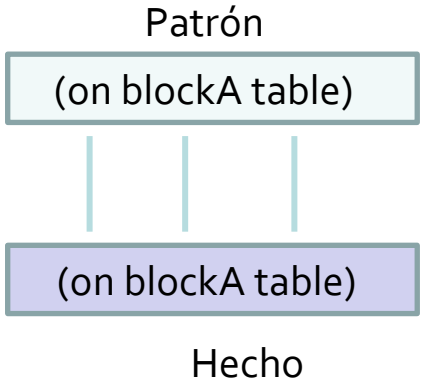
Ejemplos de patrones:

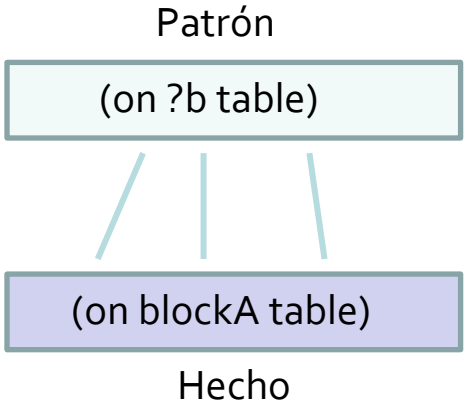
(empty)
(temperaturaAlta ?x)
(speed ?x ?y)
(on blockA ?x)
(on ?x ?y)
(tower $?  ?x  ?  $?)

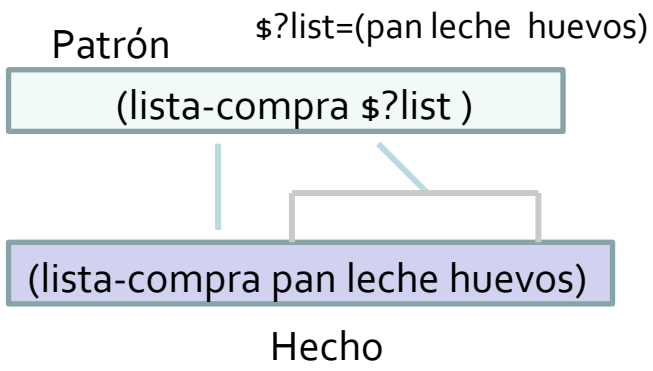


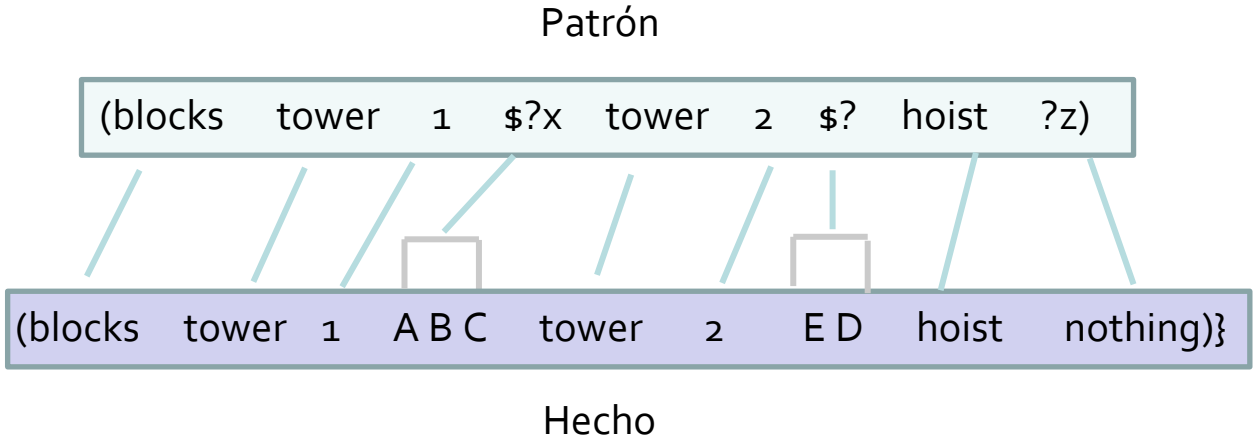


# Pattern-matching (unificación de patrones)








# Pattern matching (unificación de patrones)

Hecho

(irisflower sepal-length 6.0 sepal-width 2.2 petal-length 5.0 petal-width 1.5)

¿Y nos interesa recuperar la longitud del pétalo de la flor?

El patrón **siempre** comienza por un símbolo

Patrón

(irisflower $? petal-length ?pl $?)

Cuando dos o más variables multi-valuadas aparecen en un patrón, el patrón y el hecho pueden unificar (match) más de una vez

Patrón

(list $?x c $?y)

(list a a b a **c** b b a **c** d **c** a b **c**)

Hecho

| #Match | $?x | $?y |
|---|---|---|
| 1 | (a a b a) | (b b a c d c a b c) |
| 2 | (a a b a c b b a) | (d c a b c) |
| 3 | (a a b a c b b a c d) | (a b c) |
| 4 | (a a b a c b b a c d c a b) | () |





# Ejercicio

**Sistema basado en reglas para la seguridad de edificios**

Dada esta base de hechos:

```
(deffacts edificios
   (edificio EdificioA movimiento si horario nocturno puerta abierta)
   (edificio EdificioB movimiento si horario nocturno puerta desconocido)
   (edificio EdificioC movimiento si horario diurno puerta abierta)
)
```

Escribe los patrones de la parte izquierda de una regla CLIPS que permitan realizar las siguientes consultas:

1. Identificar si el Edificio A tiene una puerta abierta.
2. Obtener, para cada edificio, su nombre, horario y estado de la puerta.
3. Identificar los edificios en los que se ha detectado movimiento, el horario es nocturno y la puerta está abierta.
4. Identificar todos los edificios en los que se ha detectado movimiento.
5. Obtener el nombre de todos los edificios cuyo horario es nocturno.





# Ejercicio

**Sistema basado en reglas para la seguridad de edificios**

Diseña las siguientes reglas en CLIPS utilizando la BH del ejercicio anterior (es decir, hechos independientes para cada edificio). Recordemos que las reglas deben reflejar las siguientes condiciones:

- Si se detecta movimiento en un edificio durante el horario nocturno, el sistema considera que existe actividad sospechosa.
- Si existe actividad sospechosa en un edificio y alguna de sus puertas está abierta, el sistema considera que se ha producido una posible intrusión.
- Cuando se detecta una posible intrusión, el sistema debe registrar una alerta de seguridad y ejecutar la acción de activar la alarma del edificio.
- Cuando existe una alerta de seguridad en un edificio, el sistema debe ejecutar la acción de notificar al vigilante correspondiente.





# Unificación de reglas

Unificación de reglas (rule matching): la LHS de una regla unifica cuando:

- existe al menos un hecho en la BH que unifica con cada patrón de la regla
- no existe ningún hecho en la BH que unifique con los patrones *not* de la regla
- todos los tests de la regla se evalúan a TRUE

Cuando una regla unifica, se dice que la regla es aplicable y que se ha encontrado una ***instancia de la regla***

Puede haber más de una unificación (más de una instancia) para la misma regla dependiendo del número de hechos que unifiquen con cada patrón





# Semántica de test

| Test | Variables monovaluadas | Variables multivaluadas |
|---|---|---|
| (test (eq ?x c)) | ?x = c | |
| (test (not (eq ?x c)) | ?x ≠ c | |
| (test (neq ?x c)) | ?x ≠ c | |
| (test (< ?x c)) | ?x < c | |
| (test (> ?x c)) | ?x > c | |
| (test (member$ c $?x)) | | c ∈ $?x |
| (test (not (member$ c $?x)) | | c ∉ $?x |





# Unificación de reglas

## Base de Reglas

```
(defrule R1
  (servidor ?s temperatura ?t estado ?e)
  (ventilador ?s ?v)
  (test (> ?t 80))
  =>
  ...)

(defrule R2
  (servidor ?s temperatura ?t estado ?e)
  (ventilador ?s ?v)
  (test (eq ?v averiado))
  =>
  ...)

(defrule R3
  (servidor ?s temperatura ?t estado ?e)
  (test (eq ?e activo))
  =>
  ...)
```

## Base de Hechos

f-1: (servidor S1 temperatura 90 estado activo)
f-2: (ventilador S1 averiado)
f-3: (servidor S2 temperatura 65 estado activo)
f-4: (ventilador S2 operativo)
f-5: (servidor S3 temperatura 85 estado mantenimiento)

## Matching e instancias de reglas

| Rule | # Match | Facts |
|---|---|---|
| R1 | 1 | f-1, f-2: {?s=S1, ?t=90, ?e=activo, ?v=averiado} |
| | 2 | f-3, f-4: {?s=S2, ?t=65, ?e=activo, ?v=operativo} |
| R2 | 1 | f-1, f-2: {?s=S1, ?t=90, ?e=activo, ?v=averiado} |
| | 2 | f-3, f-4: {?s=S2, ?t=65, ?e=activo, ?v=operativo} |
| R3 | 1 | f-1: {?s=S1, ?t=90, ?e=activo} |
| | 2 | f-3: {?s=S2, ?t=65, ?e=activo} |
| | 3 | f-5: {?s=S3, ?t=85, ?e=mantenimiento} |

La agenda queda con 4 activaciones





# Ejercicio

```
(defrule R1
  (servidor S1)
  =>
  ...)

(defrule R2
  (servidor ?s)
  =>
  ...)

(defrule R3
  (servidor ?s1)
  (servidor ?s2)
  =>
  ...)

(defrule R4
  (alertas ?s $?antes ?a1 $?medio ?a2 $?despues)
  (test (eq ?a1 temperatura))
  (test (eq ?a2 red))
  =>
  ...)

(defrule R5
  (servidor ?s)
  (alertas ?s $?lista)
  (test (member$ ventilador $?lista))
  =>
  ...)
```

Este test se evalúa a True si 'ventilador' pertenece a la lista $?lista

**D**eterminar las instancias de cada una de las reglas que resultarán de la unificación.

```
(deffacts datos
  (servidor S1)
  (servidor S2)
  (servidor S3)
  (temperatura S1 90)
  (temperatura S2 65)
  (temperatura S3 85)
  (alertas S1 temperatura ventilador red)
)
```

Nota:
R1: 1 instancia
R2: 3 instancias
R3: 9 instancias
R4: 1 instancia
R5: 1 instancia





# Sintaxis básica de CLIPS

| Elemento | Sintaxis | Significado |
|---|---|---|
| Hecho | (temperatura S1 90) | Información de la base de hechos |
| Hechos iniciales | (deffacts iniciales ...) | Define los hechos iniciales |
| Regla | (defrule nombre ... => ...) | Define condiciones y acciones |
| Variable | ?s | Representa un valor |
| Variable multifield | $?datos | Representa cero o más valores |
| Patrón | (temperatura ?s ?t) | Busca hechos que coincidan |
| Referencia a hecho | ?f <- (temperatura ?s ?t) | Guarda una referencia al hecho |
| Condición | (test (> ?t 80)) | Evalúa una expresión |
| Negación | (not (alerta ?s)) | Comprueba la ausencia de un hecho coincidente |





# Operadores y funciones de CLIPS

| Categoría | Sintaxis | Función |
|---|---|---|
| Comparación numérica | =, <>, <, >, <=, >= | Comparar números |
| Comparación de valores | eq, neq | Comprobar igualdad o desigualdad |
| Operaciones aritméticas | +, -, *, /, mod | Realizar cálculos |
| Condiciones lógicas | and, or, not | Combinar condiciones |
| Pertenencia a lista | member$ | Comprobar pertenencia |
| Añadir hechos | (assert (alerta S1)) | Añadir hechos a la BH |
| Eliminar hechos | (retract ?f) | Eliminar hechos de la BH |
| Mostrar información | (printout t "Alerta" crlf) | Mostrar un mensaje |

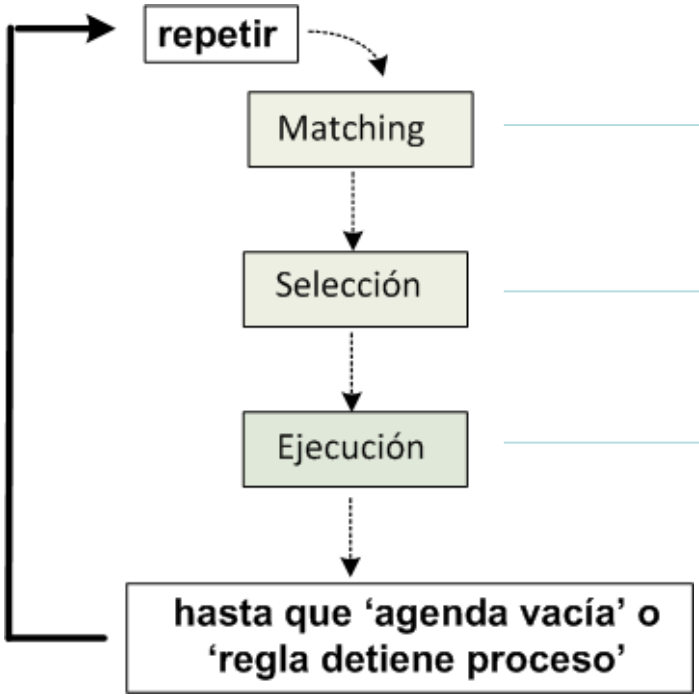




# Motor de inferencia

**CLIPS utiliza un motor de inferencia hacia delante** basado en el algoritmo de matching Rete.

El mecanismo de control que siguen los motores de inferencia hacia delante se denomina **ciclo reconocimiento-acción** (también ciclo matching-selección-acción):




Matching de las reglas: genera un Conjunto conflicto (o Agenda) con todas las reglas aplicables o instancias de reglas encontradas

Selecciona una instancia del conjunto Conflicto de acuerdo a un criterio predeterminado (Estrategia de Resolución de Conflictos)

Se ejecuta la RHS de la instancia seleccionada, actualizando la BH y/o ejecutando las acciones externas





# Motor de inferencia

BH = base de hechos; BR= base de reglas; CC= conjunto conflicto; InstRule=instancia de regla;

```
BH=hechos inciales
CC=Matching(BH,BR)
while objetivo ⊄ BH y CC ≠ ∅
    InstRule=Seleccionar(CC)                    ;; Se elimina InstRule de CC
    BH=BH \ hechos eliminados por InstRule      ;; Ejecución de la RHS (1)
    BH=BH ∪ hechos añadidos por InstRule        ;; Ejecución de la RHS (2)
    CC=CC \ instancias de reglas eliminadas resultado de hechos eliminados (3)
    CC=CC ∪ Matching(BH,BR)                                                 (4)
end_while
if objetivo ⊆ BH        ;; disparo de una regla que encuentra el objetivo y para el MI
    then EXITO
    else FALLO
```

**(1) (2):** ejecución de la RHS de InstRule ; ejecutar los comandos 'retract' y 'assert' y actualizar la BH en consonancia

**(3)** Actualizar el CC las instancias que dependen de los hechos eliminados (retract)

**(4)** Activación de la fase de matching con los nuevos hechos generados (assert)

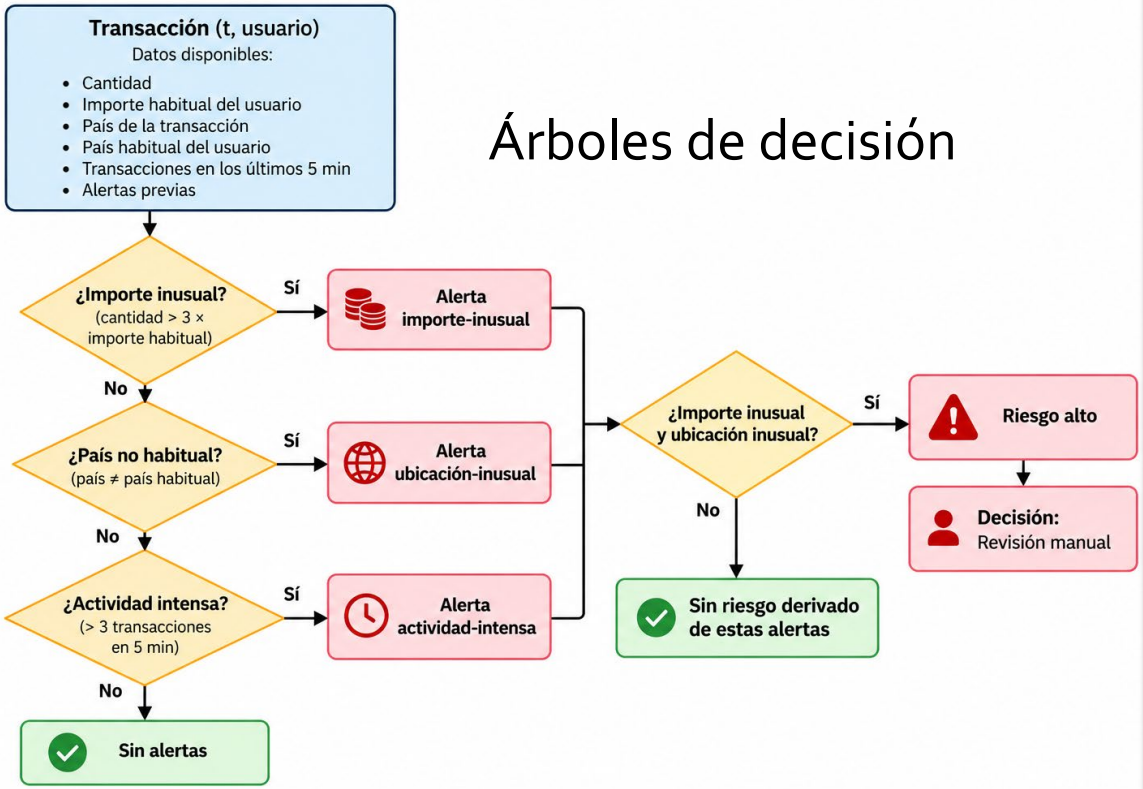




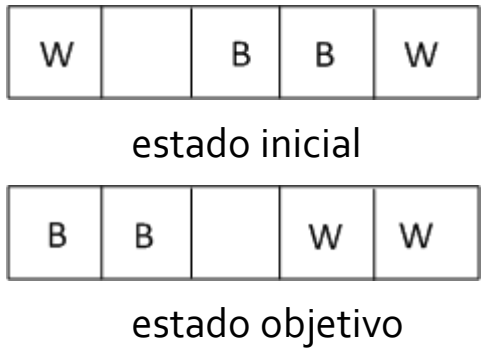
# Tipos de problemas resolubles con un SBR

Nos vamos a centrar en los problemas en los que un SBR presenta una diferencia significativa respecto a otras aproximaciones ya estudiadas (lógica y ontologías):

## Árboles de decisión



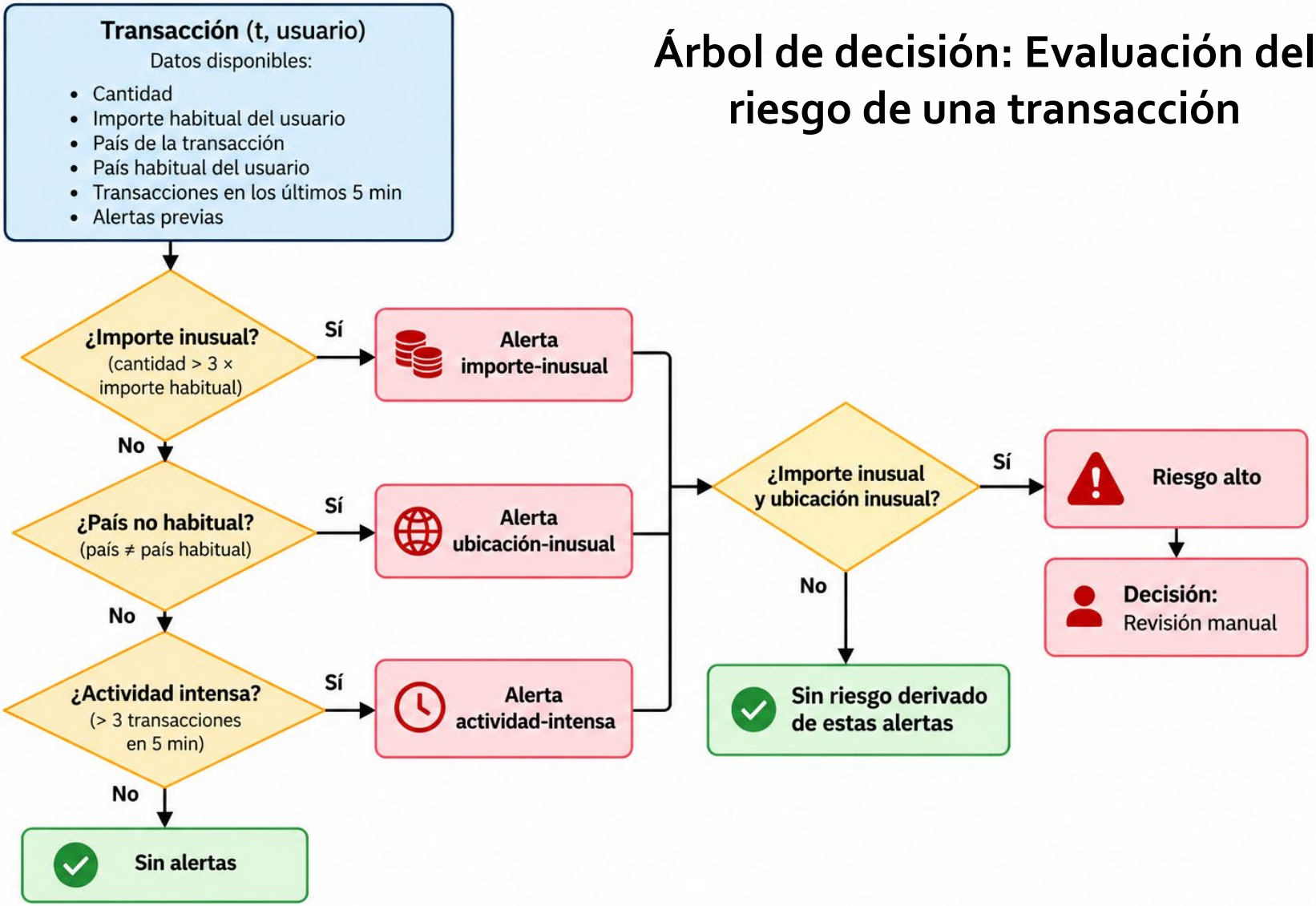

## Búsqueda de soluciones






# Ejemplo: motor de inferencia

## Árbol de decisión: Evaluación del riesgo de una transacción



# Ejemplo: motor de inferencia

```
(defrule detectar-importe-inusual
  (transaccion ?t ?u)
  (cantidad ?t ?c)
  (importe-habitual ?u ?h)
  (test (> ?c (* 3 ?h)))
=>
 (assert (alerta ?t importe-inusual)))

(defrule detectar-ubicacion-rara
  (transaccion ?t ?u)
  (pais ?t ?p)
  (pais-habitual ?u ?ph)
  (test (neq ?p ?ph))
=>
  (assert (alerta ?t ubicacion-rara)))

(defrule detectar-actividad-intensa
  (transaccion ?t ?u)
  (transacciones-ultimos-5min ?n)
  (test (> ?n 3))
=>
  (assert (alerta ?t actividad-intensa)))
```

```
(defrule riesgo-alto-tipo1
  (alerta ?t importe-inusual)
  (alerta ?t ubicacion-rara)
=>
  (assert (riesgo ?t alto)))

(defrule riesgo-alto-tipo2
  (alerta ?t actividad-intensa)
  (alertas-previas si)
=>
  (assert (riesgo ?t alto)))

(defrule revision-manual
  (riesgo ?t alto)
  (transaccion ?t ?u)
=>
  (assert (decision ?t revision-manual))
  (printout t "Revisar transaccion " ?t " del usuario " ?u crlf)))
```





# Ejemplo: motor de inferencia

```
(deffacts inicio
   (transaccion T1 user2658)
   (cantidad T1 1500)
   (importe-habitual user2658 200)
   (pais T1 Brasil)
   (pais-habitual user2658 España)
   (transacciones-ultimos-5min T1 5)
   (alertas-previas no)
 )
```

## Base de hechos

f-1: (transaccion T1 user2658)
f-2 (cantidad T1 1500)
f-3: (importe-habitual user2658 200)
f-4: (país T1  Brasil)
f-5: (país-habitual user2658 España)
f-6: (transacciones-últimos-5min T1 5)
f-7: (alertas-previas no)

## Matching

detectar_importe_inusual: f-1, f-2, f-3 - ?t=T1, ?u=user2658, ?c=1500, ?h=200
detectar_ubicacion_rara: f-1, f-4, f-5 - ?t=T1, ?u=user2658, ?p=Brasil, ?ph=España
detectar_actividad_intensa: f-1, f-6 - ?t=T1, ?u=user2658, ?n=5

Agenda o Conjunto Conflicto
(instancias de reglas)





# Ejemplo: motor de inferencia

## Selección

detectar_importe_inusual: f-1, f-2, f-3 - ?t=T1, ?u=user2658, ?c=1500, ?h=200
detectar_ubicacion_rara: f-1, f-4, f-5 - ?t=T1, ?u=user2658, ?p=Brasil, ?ph=España
detectar_actividad_intensa: f-1, f-6 - ?t=T1, ?u=user2658, ?n=5

## Ejecución

Base de hechos

f-1: (transaccion T1 user2658)
f-2 (cantidad T1 1500)
f-3: (importe-habitual user2658 200)
f-4: (país T1 Brasil)
f-5: (país-habitual user2658 España)
f-6: (transacciones-últimos-5min T1 5)
f-7: (alertas-previas no)
**f-8: (alerta T1 importe-inusual)**

## Matching

detectar_ubicacion_rara: f-1, f-4, f-5 - ?t=T1, ?u=user2658, ?p=Brasil, ?ph=España
detectar_actividad_intensa: f-1, f-6 - ?t=T1, ?u=user2658, ?n=5

## Selección

detectar_ubicacion_rara: f-1, f-4, f-5 - ?t=T1, ?u=user2658, ?p=Brasil, ?ph=España
detectar_actividad_intensa: f-1, f-6 - ?t=T1, ?u=user2658, ?n=5

## Ejecución

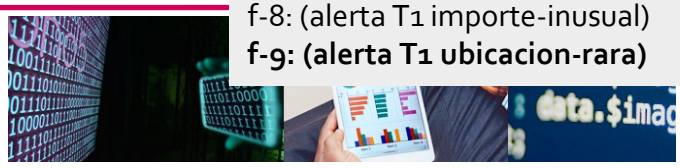
Base de hechos

f-1: (transaccion T1 user2658)
f-2 (cantidad T1 1500)
f-3: (importe-habitual user2658 200)
f-4: (país T1 Brasil)
f-5: (país-habitual user2658 España)
f-6: (transacciones-últimos-5min T1 5)
f-7: (alertas-previas no)
f-8: (alerta T1 importe-inusual)
**f-9: (alerta T1 ubicacion-rara)**

## Matching

detectar_actividad_intensa: f-1, f-6 - ?t=T1, ?u=user2658, ?n=5
riesgo_alto_tipo1: f-8, f-9: ?t=T1





# Ejemplo: motor de inferencia

## Selección

detectar_actividad_intensa: f-1, f-6 - ?t=T1, ?u=user2658, ?n=5
riesgo_alto_tipo1: f-8, f-9: ?t=T1

## Ejecución

Base de hechos

f-1: (transaccion T1 user2658)
f-2 (cantidad T1 1500)
f-3: (importe-habitual user2658 200)
f-4: (país T1 Brasil)
f-5: (país-habitual user2658 España)
f-6: (transacciones-últimos-5min T1 5)
f-7: (alertas-previas no)
f-8: (alerta T1 importe-inusual)
f-9: (alerta T1 ubicacion-rara)
**f-10: (alerta T1 actividad-intensa)**

## Matching

riesgo_alto_tipo1: f-8, f-9: ?t=T1

## Selección

riesgo_alto_tipo1: f-8, f-9: ?t=T1

## Ejecución

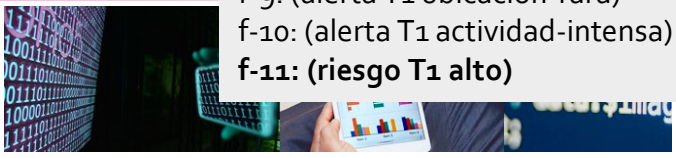
Base de hechos

f-1: (transaccion T1 user2658)
f-2 (cantidad T1 1500)
f-3: (importe-habitual user2658 200)
f-4: (país T1 Brasil)
f-5: (país-habitual user2658 España)
f-6: (transacciones-últimos-5min T1 5)
f-7: (alertas-previas no)
f-8: (alerta T1 importe-inusual)
f-9: (alerta T1 ubicacion-rara)
f-10: (alerta T1 actividad-intensa)
**f-11: (riesgo T1 alto)**

## Matching

revision_manual: f-1, f-11: ?t=T1, ?u=user2658





# Ejemplo: motor de inferencia

## Selección

revision_manual: f-1, f-11: ?t=T1, ?u=user2658

## Ejecución

Base de hechos

f-1: (transaccion T1 user2658)
f-2 (cantidad T1 1500)
f-3: (importe-habitual user2658 200)
f-4: (país T1 Brasil)
f-5: (país-habitual user2658 España)
f-6: (transacciones-últimos-5min T1 5)
f-7: (alertas-previas no)
f-8: (alerta T1 importe-inusual)
f-9: (alerta T1 ubicacion-rara)
f-10: (alerta T1 actividad-intensa)
f-11: (riesgo T1 alto)
**f-12: (decision T1 revision-manual)**

Revisar transaccion T1 del usuario user2658

## Matching

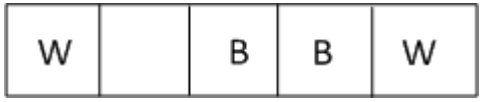
Agenda vacía, finaliza la ejecución del SBR





# Ejemplo: motor de inferencia

## Búsqueda de soluciones - Puzzle lineal:




estado inicial




estado objetivo

**Base de Reglas**

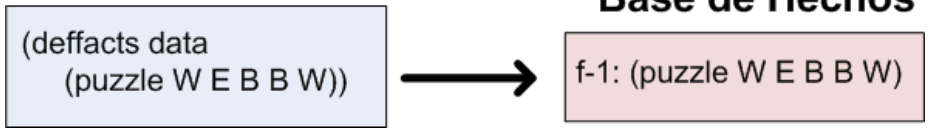
```
(defrule emtpy-1-left
    (puzzle $?x ?y E $?z)
  =>
    (assert (puzzle $?x E ?y $?z)))

(defrule emtpy-2-left
    (puzzle $?x ?y1 ?y2 E $?z)
  =>
    (assert (puzzle $?x E ?y2 ?y1 $?z)))

(defrule emtpy-1-right
    (puzzle $?x E ?y $?z)
  =>
    (assert (puzzle $?x ?y E $?z)))

(defrule emtpy-2-right
    (puzzle $?x E ?y1 ?y2 $?z)
  =>
    (assert (puzzle $?x ?y2 ?y1 E $?z)))
```

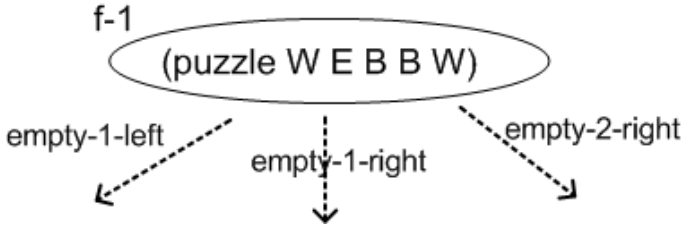
**Base de Hechos**




## Matching

empty-1-left: f-1 ?y=W, $?x=(), $?z=(B B W)
empty-1-right: f-1 ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1 ?y1=B, ?y2=B, $?x=(W), $?z=(W)

Agenda ó Conjunto Conflicto (instancias de reglas)








# Ejemplo: motor de inferencia

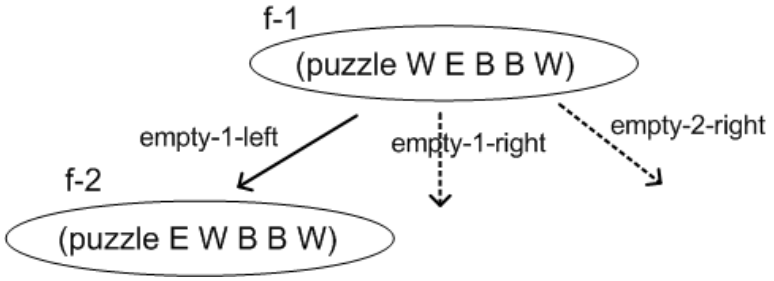
**empty-1-left: f-1 ?y=W, $?x=(), $?z=(B B W)**
empty-1-right: f-1 ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1 ?y1=B, ?y2=B, $?x=(W), $?z=(W)

→ **Selección**

## Ejecución

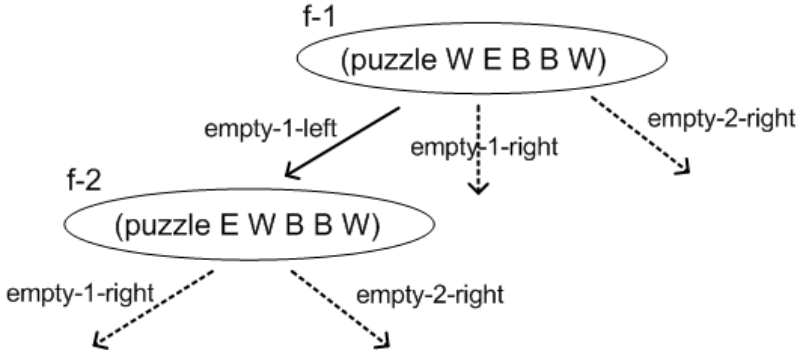
**Base de Hechos**

f-1: (puzzle W E B B W)
f-2: (puzzle E W B B W)




## Matching

empty-1-right: f-1, ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1, ?y1=B, ?y2=B, $?x=(W), $?z=(W)
**empty-1-right: f-2, ?y=W, $?x=(), $?z=(B B W)**
**empty-2-right: f-2, ?y1=W, ?y2=B, $?x=(), $?Z=(B W)**








# Ejemplo: motor de inferencia

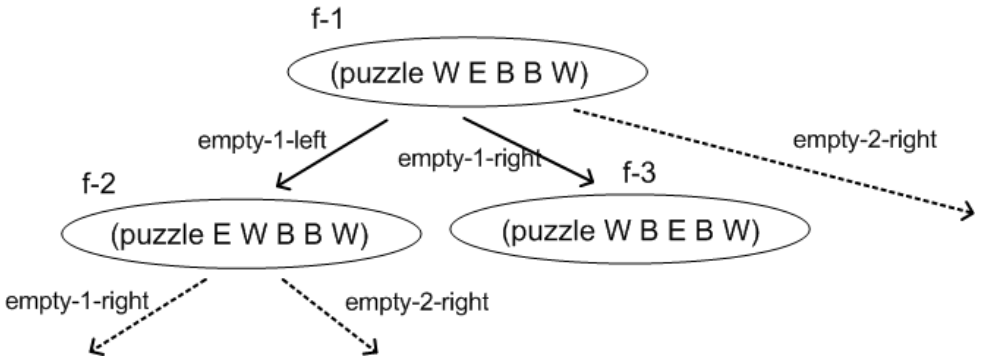
**empty-1-right: f-1, ?y=B, $?x=(W), $?z=(B W)**
empty-2-right: f-1, ?y1=B, ?y2=B, $?x=(W), $?z=(W)
empty-1-right: f-2, ?y=W, $?x=(), $?z=(B B W)
empty-2-right: f-2, ?y1=W, ?y2=B, $?x=(), $?Z=(B W)

## Selección

## Ejecución

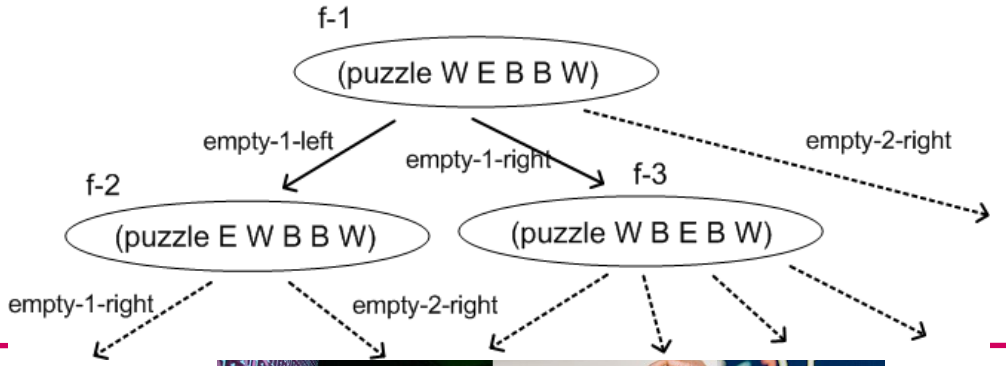
**Base de Hechos**

f-1: (puzzle W E B B W)
f-2: (puzzle E W B B W)
f-3: (puzzle W B E B W)




## Matching

empty-2-right: f-1, ?y1=B, ?y2=B, $?x=(W), $?z=(W)
empty-1-right: f-2, ?y=W, $?x=(), $?z=(B B W)
empty-2-right: f-2, ?y1=W, ?y2=B, $?x=(), $?Z=(B W)
**empty-1-left: f-3, …**
**empty-2-left: f-3, …**
**empty-1-right: f-3, …**
**empty-2-right: f-3, …**






# Estrategias de resolución de conflictos

El Conjunto Conflicto o Agenda es una colección de instancias de reglas o activaciones que son el resultado de hacer matching entre las reglas de la BR y los hechos de la BH. En la agenda puede haber cero o más instancias de reglas.

## Refracción

Una regla solo se puede disparar una vez con los mismos hechos (hechos con los mismos índices) . Cuando la BH se modifica, todas las reglas se pueden volver a utilizar de nuevo siempre que se haya producido al menos un nuevo hecho que genere una nueva activación de la regla.

CLIPS aplica refracción, no permitiendo que una regla se dispare dos veces con los mismos datos, evitando así que se disparen repetidamente reglas con los mismos hechos (bucle infinito).

Este comportamiento se complementa con la opción por defecto de CLIPS de no permitir duplicidad de hechos. CLIPS no acepta hechos duplicados, es decir, hechos idénticos pero con diferentes índices (este comportamiento se puede, no obstante, anular seleccionado otra opción).





# Ejemplo: refracción

**empty-1-left: f-1 ?y=W, $?x=(), $?z=(B B W)**
empty-1-right: f-1 ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1 ?y1=B, ?y2=B, $?x=(W), $?z=(W)

**Base de Hechos**

f-1: (puzzle W E B B W)
f-2: (puzzle E W B B W)

## Matching

empty-1-right: f-1, ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1, ?y1=B, ?y2=B, $?x=(W), $?z=(W)
**empty-1-right: f-2, ?y=W, $?x=(), $?z=(B B W)**
**empty-2-right: f-2, ?y1=W, ?y2=B, $?x=(), $?Z=(B W)**

La regla empty-1-left no se vuelve a instanciar con el hecho f-1 porque esta instanciación ya se generó en el ciclo anterior.

Si se selecciona empty-1-right para su ejecución, se genera el hecho (puzzle W E B B W) que es un hecho idéntico a f-1. Por tanto, este hecho no se incluye en la BH.

Una misma activación (regla + índice de hechos) solo se inserta una vez en la Agenda.
Un mismo hecho solo se inserta una vez en la BH.





# Estrategias de resolución de conflictos

El criterio para la **resolución de conflictos** es fundamental para una eficiente resolución del problema

1. La elección de una estrategia de resolución de conflictos distinta puede conducir a una ejecución distinta del SBR
2. Las reglas interaccionan entre sí por lo que el orden en el que éstas se disparan es determinante en la ejecución del SBR
3. Es importante no solo realizar un buen diseño de las reglas sino tener en cuenta cuando éstas se van a disparar y las relaciones que surgen entre las reglas.

Cuando hay más de una instancia en la Agenda, la **estrategia de resolución de conflictos** decide la activación a escoger, de modo que la instancia que aparece en primer lugar en la agenda es siempre la instancia que se selecciona para su ejecución.





# Estrategias de resolución de conflictos

## Profundidad

Las nuevas activaciones se manejan como una cola de prioridades. Los empates se resuelven mediante una estrategia LIFO (más prioridad para los hechos e instancias más recientes).
Es la estrategia por defecto.

## Matching (profundidad)

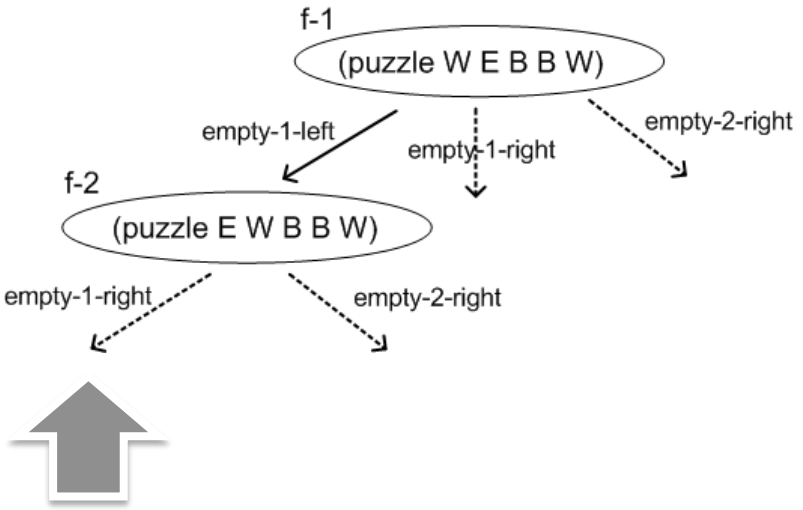
**empty-1-right: f-2, ?y=W, $?x=(), $?z=(B B W)**
**empty-2-right: f-2, ?y1=W, ?y2=B, $?x=(), $?Z=(B W)**
empty-1-right: f-1, ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1, ?y1=B, ?y2=B, $?x=(W), $?z=(W)

## Agenda

**empty-1-left: f-1 ?y=W, $?x=(), $?z=(B B W)**
empty-1-right: f-1 ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1 ?y1=B, ?y2=B, $?x=(W), $?z=(W)

**Base de Hechos**

f-1: (puzzle W E B B W)
f-2: (puzzle E W B B W)








# Estrategias de resolución de conflictos

## Anchura

Las nuevas activaciones se manejan como una cola de prioridades. Los empates se resuelven mediante una estrategia FIFO (más prioridad para los hechos e instancias más antiguos).

## Matching (anchura)

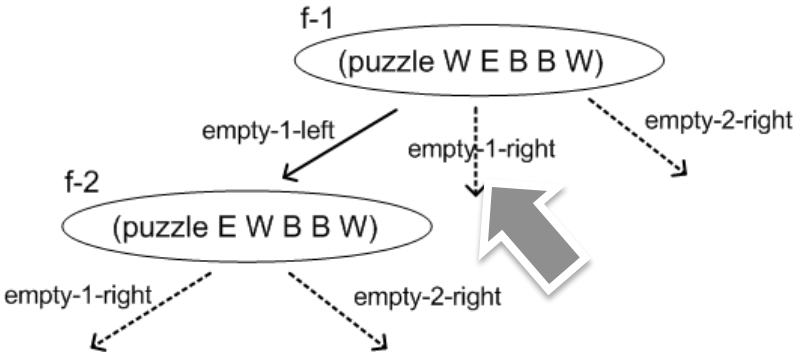
empty-1-right: f-1, ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1, ?y1=B, ?y2=B, $?x=(W), $?z=(W)
**empty-1-right: f-2, ?y=W, $?x=(), $?z=(B B W)**
**empty-2-right: f-2, ?y1=W, ?y2=B, $?x=(), $?Z=(B W)**

## Agenda

**empty-1-left: f-1 ?y=W, $?x=(), $?z=(B B W)**
empty-1-right: f-1 ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1 ?y1=B, ?y2=B, $?x=(W), $?z=(W)

**Base de Hechos**

f-1: (puzzle W E B B W)
f-2: (puzzle E W B B W)








# Estrategias de resolución de conflictos

## Aleatoria

A cada activación se le asigna un número aleatorio que se usa para determinar la posición de la activación en la agenda. CLIPS permite seleccionar Random como estrategia de resolución de conflictos.

## Matching (aleatoria)

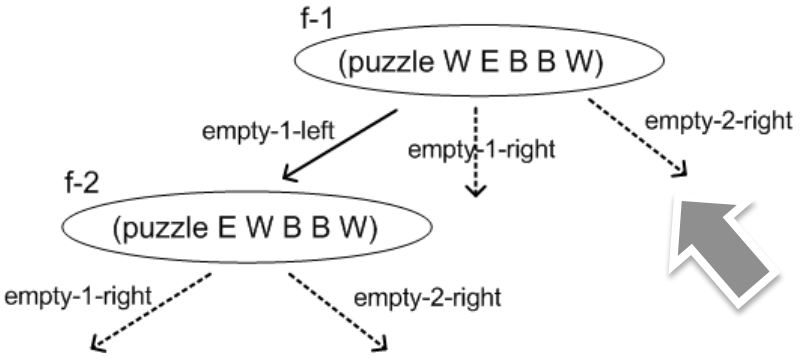
empty-2-right: f-1, ?y1=B, ?y2=B, $?x=(W), $?z=(W)
**empty-1-right: f-2, ?y=W, $?x=(), $?z=(B B W)**
empty-1-right: f-1, ?y=B, $?x=(W), $?z=(B W)
**empty-2-right: f-2, ?y1=W, ?y2=B, $?x=(), $?Z=(B W)**

## Agenda

**empty-1-left: f-1 ?y=W, $?x=(), $?z=(B B W)**
empty-1-right: f-1 ?y=B, $?x=(W), $?z=(B W)
empty-2-right: f-1 ?y1=B, ?y2=B, $?x=(W), $?z=(W)

**Base de Hechos**

f-1: (puzzle W E B B W)
f-2: (puzzle E W B B W)








# Estrategias de resolución de conflictos

## Prioridades explícitas

- Independientemente de la estrategia escogida, se puede asignar un valor de prioridad a cada regla. Este valor se denomina **salience**.
- En CLIPS, el rango del salience va desde -10,000 hasta 10,000.
- Cuanto mayor sea el valor, más alta es la prioridad de la regla. Es decir, la agenda se ordena, en primer lugar, en función de esta prioridad.
- Si no se especifica salience en la regla, entonces su prioridad es 0.

Añadimos una regla que detecte que hemos alcanzado una solución:

**Base de Reglas**

```
(defrule emtpy-1-left
    (puzzle $?x ?y E $?z)
  =>
    (assert (puzzle $?x E ?y $?z)))

(defrule emtpy-2-left
    (puzzle $?x ?y1 ?y2 E $?z)
  =>
    (assert (puzzle $?x E ?y2 ?y1 $?z)))

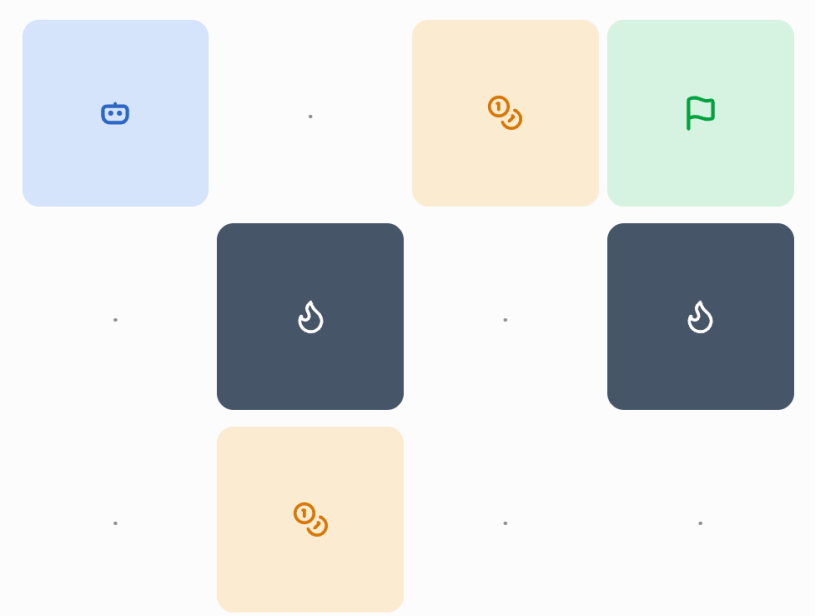
(defrule emtpy-1-right
    (puzzle $?x E ?y $?z)
  =>
    (assert (puzzle $?x ?y E $?z)))

(defrule emtpy-2-right
    (puzzle $?x E ?y1 ?y2 $?z)
  =>
    (assert (puzzle $?x ?y2 ?y1 E $?z)))
```

```
(defrule goal
    (declare (salience 100))
    (puzzle B B E W W)
=>
    (printout t "Solución alcanzada" crlf)
    (halt)
)
```



# Escenario



Coordenadas (fila, columna). Inicio: (1,1). Salida: (1,4).
Monedas: (1,3) y (3,2). Peligros: (2,2) y (2,4).

El robot conoce las conexiones del grid, pero no dispone de ninguna heurística que le indique qué celda está más cerca de la salida. Su objetivo principal es encontrarla; recoger monedas es un objetivo secundario.

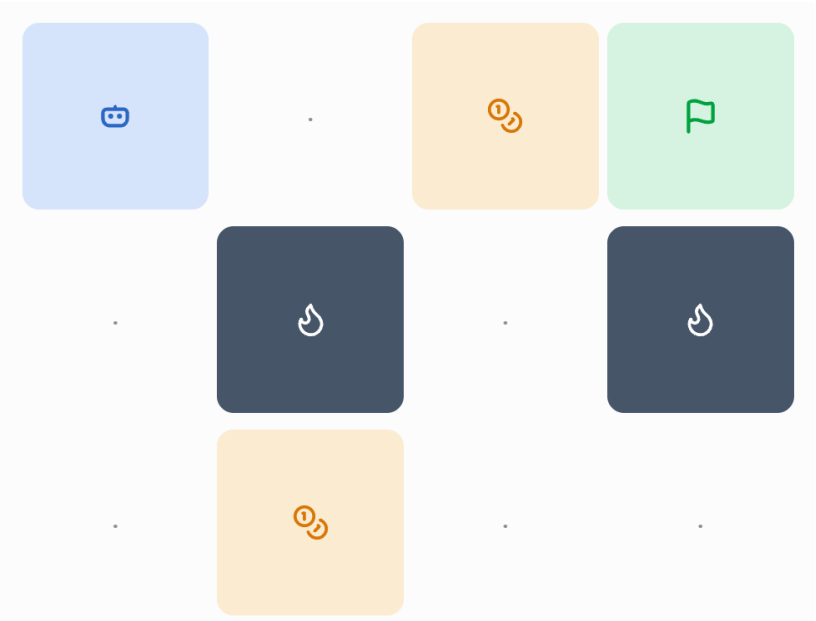
Utilizaremos los siguientes tipos de hechos:

- (posicion F C): posición física actual.
- (conexion F1 C1 F2 C2): movimiento permitido entre dos celdas.
- (visitada F C): celda ya explorada.
- (moneda F C), (salida F C) y (peligro F C): elementos del juego.
- (puntos P): monedas recogidas.





# Ejercicio



Coordenadas (fila, columna). Inicio: (1,1). Salida: (1,4).
Monedas: (1,3) y (3,2). Peligros: (2,2) y (2,4).

Dados los tipos de hechos anteriores, diseña la base de hechos inicial





# Reglas

```
(defrule encontrar-salida
  (declare (salience 100))
  (posicion ?f ?c)
  (salida ?f ?c)
=>
  (printout t "Salida encontrada" crlf)
  (halt)
)

(defrule recoger-moneda
  (declare (salience 50))
  (posicion ?f ?c)
  ?m <- (moneda ?f ?c)
  ?p <- (puntos ?n)
=>
  (retract ?m ?p)
  (assert (puntos (+ ?n 1)))
  (printout t "Moneda recogida en " ?f "," ?c crlf)
)
```

**Ejercicio**: diseña la regla mover





# Ejemplo: prioridades explícitas

Base de hechos actual:

...
f-20 (posicion 1 3)
f-21 (moneda 1 3)
f-22 (puntos 0)
f-23 (conexion 1 3 1 4)
f-24 (conexion 1 3 2 3)
...

```
(defrule recoger-moneda
  (declare (salience 50))
  (posicion ?f ?c)
  ?m <- (moneda ?f ?c)
  ?p <- (puntos ?n)
=>
  (retract ?m ?p)
  (assert (puntos (+ ?n 1)))
  (printout t "Moneda recogida en " ?f "," ?c crlf)
)
```

Agenda:

| Salience | Activación |
|---|---|
| 50 | recoger-moneda: f-20, f-21, f-22 |
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (2,3) |

Nueva base de hechos:

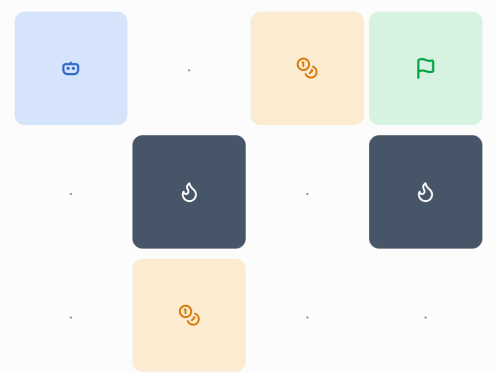
...
f-20 (posicion 1 3)
~~f-21 (moneda 1 3)~~
~~f-22 (puntos 0)~~
f-23 (conexion 1 3 1 4)
f-24 (conexion 1 3 2 3)
...
f-50 (puntos 1)

Las prioridades explícitas permiten seleccionar primero las acciones más importantes. Cuando varias activaciones tienen el mismo salience, es necesaria una estrategia adicional de resolución de conflictos (anchura o profundidad).



# Ejemplo: anchura

Base de hechos inicial:
- f-1: (posicion 1 1)
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- ...




```
(printout t "Mover a " ?nf "," ?nc crlf)
)
```

Se asume que se crean primero las activaciones que visitan celdas en la misma fila

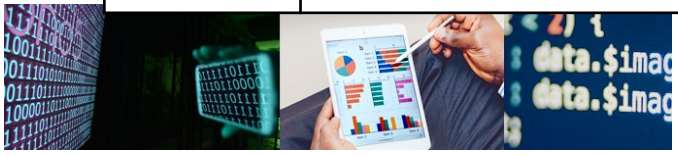
| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,2) |
| 0 | mover: hacia (2,1) |

Nueva base de hechos:
~~f-1: (posicion 1 1)~~
f-2: (visitada 1 1)
...
f-29: (posicion 1 2)
f-30: (visitada 1 2)

Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,2) |
| 0 | mover: hacia (2,1) |
| 0 | mover: hacia (1,3) |





# Ejemplo: anchura

Base de hechos actual:
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- ...
- f-29: (posicion 1 2)
- f-30: (visitada 1 2)

```
(defrule mover
  (declare (salience 0))
  ?p <- (posicion ?f ?c)
  (conexion ?f ?c ?nf ?nc)
  (not (visitada ?nf ?nc))
  (not (peligro ?nf ?nc))
=>
  (retract ?p ?s)
  (assert (posicion ?nf ?nc))
  (assert (visitada ?nf ?nc))
  (printout t "Mover a " ?nf "," ?nc crlf)
)
```

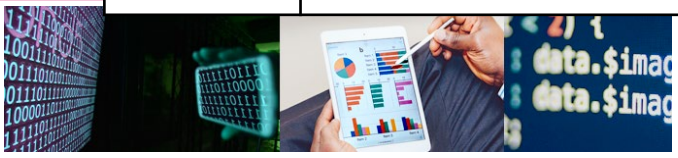
Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,3) |
| | |

Nueva base de hechos:
...
~~f-29: (posicion 1 2)~~
f-30: (visitada 1 2)
f-31: (posicion 1 3)
f-32: (visitada 1 3)

Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 50 | recoger-moneda (1,3) |
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (2,3) |



# Ejemplo: anchura

Base de hechos actual:
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- …
- f-30: (visitada 1 2)
- f-31: (posicion 1 3)
- f-32: (visitada 1 3)

```
(defrule recoger-moneda
  (declare (salience 50))
  (posicion ?f ?c)
  ?m <- (moneda ?f ?c)
  ?p <- (puntos ?n)
=>
  (retract ?m ?p)
  (assert (puntos (+ ?n 1)))
  (printout t "Moneda recogida en "
?f "," ?c crlf)
)
```

Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 50 | recoger-moneda (1,3) |
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (2,3) |

Nueva base de hechos:
~~f-3: (puntos 0)~~ ~~f-4: (moneda 1 3)~~
…
f-31: (posicion 1 3)
f-32: (visitada 1 3)
f-33: (puntos 1)

Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 50 | recoger-moneda (1,3) |
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (2,3) |





# Ejemplo: anchura

Base de hechos actual:

f-2: (visitada 1 1)
f-5: (moneda 3 2)
f-6: (peligro 2 2)
f-7: (peligro 2 4)
f-8: (salida 1 4)
f-9: (conexion 1 1 1 2)
f-10: (conexion 1 1 2 1)
f-11: (conexion 1 2 1 1)
f-12: (conexion 1 2 1 3)
f-13: (conexion 1 3 1 2)
f-14: (conexion 1 3 1 4)
f-15: (conexion 1 3 2 3)
...
f-30: (visitada 1 2)
f-31: (posicion 1 3)
f-32: (visitada 1 3)
f-33: (puntos 1)

Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (2,3) |

Nueva base de hechos:

...
~~f-31: (posicion 1 3)~~
f-32: (visitada 1 3)
f-33: (puntos 1)
f-34: (posicion 1 4)
f-35: (visitada 1 4)

Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (2,3) |
| 100 | encontrar-salida |





# Ejemplo: anchura

Base de hechos actual:
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- ...
- f-30: (visitada 1 2)
- f-32: (visitada 1 3)
- f-33: (puntos 1)
- f-34: (posicion 1 4)
- f-35: (visitada 1 4)

```
(defrule encontrar-salida
  (declare (salience 100))
  (posicion ?f ?c)
  (salida ?f ?c)
=>
  (printout t "Salida encontrada" crlf)
  (halt)
)
```

Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 100 | encontrar-salida |
| | |

La base de hechos no cambia:
- ...
- f-32: (visitada 1 3)
- f-33: (puntos 1)
- f-34: (posicion 1 4)
- f-35: (visitada 1 4)

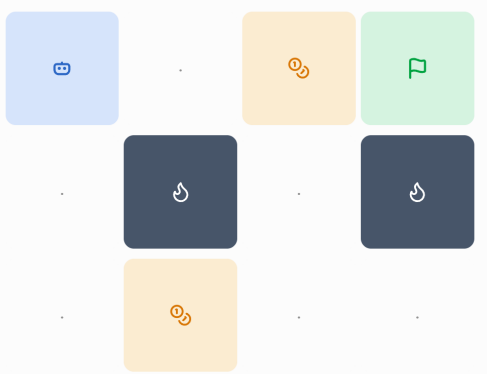
**Finaliza la ejecución**





# Ejemplo: profundidad

Base de hechos inicial:
- f-1: (posicion 1 1)
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- …




Se asume que se crean primero las activaciones que visitan celdas en la misma fila

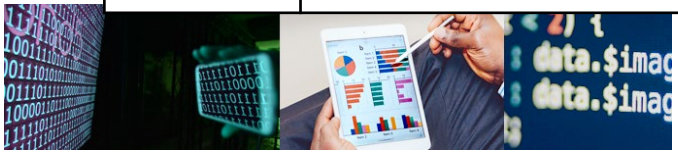
| Salience | Activación |
|---|---|
| 0 | mover: hacia (2,1) |
| 0 | mover: hacia (1,2) |

Nueva base de hechos:
- ~~f-1: (posicion 1 1)~~
- f-2: (visitada 1 1)
- …
- f-29: (posicion 2 1)
- f-30: (visitada 2 1)

Se actualiza la agenda:

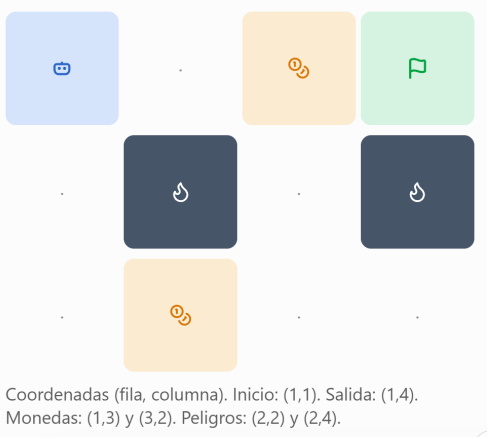
| Salience | Activación |
|---|---|
| 0 | mover: hacia (2,1) |
| 0 | mover: hacia (1,2) |
| 0 | mover: hacia (3,1) |





# Ejemplo: profundidad

Base de hechos inicial:
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- …
- f-29: (posicion 2 1)
- f-30: (visitada 2 1)




Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 0 | mover: hacia (3,1) |
| | |

Nueva base de hechos:
…
~~f-29: (posicion 2 1)~~
f-30: (visitada 2 1)
f-31: (posicion 3 1)
f-32: (visitada 3 1)

Se actualiza la agenda:

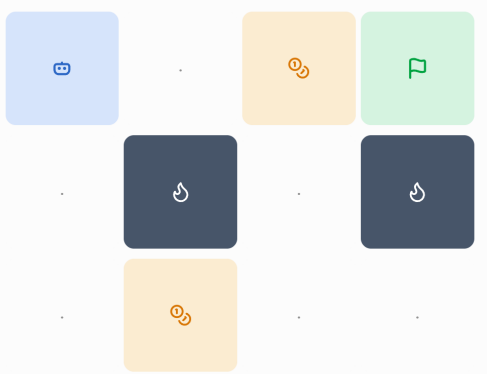
| Salience | Activación |
|---|---|
| 0 | mover: hacia (3,1) |
| 0 | mover: hacia (3,2) |





# Ejemplo: profundidad

Base de hechos inicial:
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- …
- f-30: (visitada 2 1)
- f-31: (posicion 3 1)
- f-32: (visitada 3 1)



Coordenadas (fila, columna). Inicio: (1,1). Salida: (1,4).
Monedas: (1,3) y (3,2). Peligros: (2,2) y (2,4).

Se asume que se crean primero las activaciones que visitan celdas en la misma fila

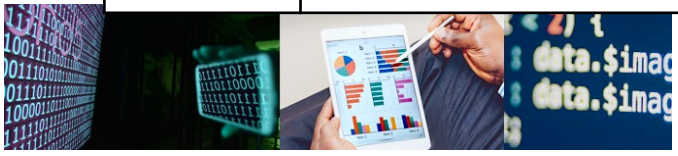
| Salience | Activación |
|---|---|
| 0 | mover: hacia (3,2) |
| | |

Nueva base de hechos:
…
~~f-31: (posicion 3 1)~~
f-32: (visitada 3 1)
f-33: (posicion 3 2)
f-34: (visitada 3 2)

Se actualiza la agenda:

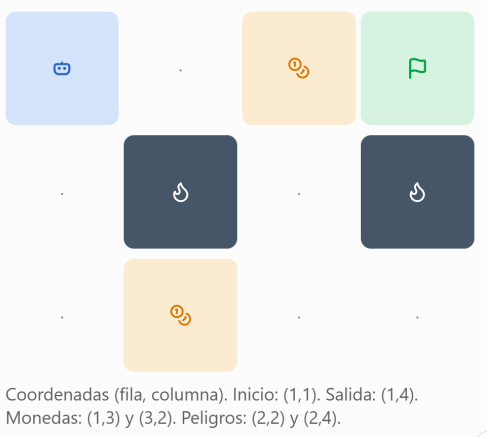
| Salience | Activación |
|---|---|
| 0 | mover: hacia (3,2) |
| 50 | recoger-moneda (3,2) |
| 0 | mover: hacia (3,3) |





# Ejemplo: profundidad

Base de hechos inicial:
- f-2: (visitada 1 1)
- f-3: (puntos 0)
- f-4: (moneda 1 3)
- f-5: (moneda 3 2)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- …
- f-30: (visitada 2 1)
- f-32: (visitada 3 1)
- f-33: (posicion 3 2)
- f-34: (visitada 3 2)



Coordenadas (fila, columna). Inicio: (1,1). Salida: (1,4).
Monedas: (1,3) y (3,2). Peligros: (2,2) y (2,4).

Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 50 | recoger-moneda (3,2) |
| 0 | mover: hacia (3,3) |

Nueva base de hechos:
~~f-3: (puntos 0)~~ ~~f-5: (moneda 3 2)~~
…
f-33: (posicion 3 2)
f-34: (visitada 3 2)
f-35: (puntos 1)

Se actualiza la agenda:

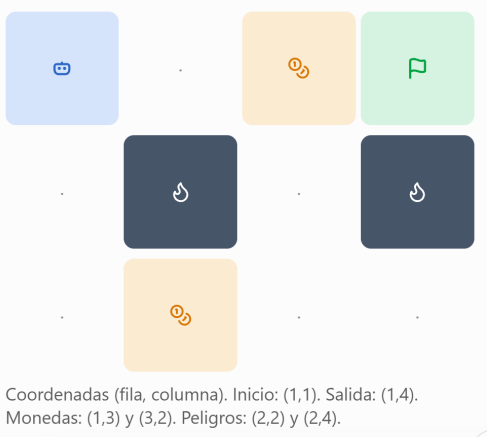
| Salience | Activación |
|---|---|
| 50 | recoger-moneda (3,2) |
| 0 | mover: hacia (3,3) |





# Ejemplo: profundidad

Base de hechos inicial:
- f-2: (visitada 1 1)
- f-4: (moneda 1 3)
- f-6: (peligro 2 2)
- f-7: (peligro 2 4)
- f-8: (salida 1 4)
- f-9: (conexion 1 1 1 2)
- f-10: (conexion 1 1 2 1)
- f-11: (conexion 1 2 1 1)
- f-12: (conexion 1 2 1 3)
- f-13: (conexion 1 3 1 2)
- f-14: (conexion 1 3 1 4)
- f-15: (conexion 1 3 2 3)
- ...
- f-30: (visitada 2 1)
- f-32: (visitada 3 1)
- f-33: (posicion 3 2)
- f-34: (visitada 3 2)
- f-35: (puntos 1)




Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 0 | mover: hacia (3,3) |
| | |

Nueva base de hechos:
...
~~f-33: (posicion 3 2)~~
f-34: (visitada 3 2)
f-35: (puntos 1)
f-36: (posicion 3 3)
f-37: (visitada 3 3)

Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 0 | mover: hacia (3,3) |
| 0 | mover: hacia (2,3) |
| 0 | mover: hacia (3,4) |

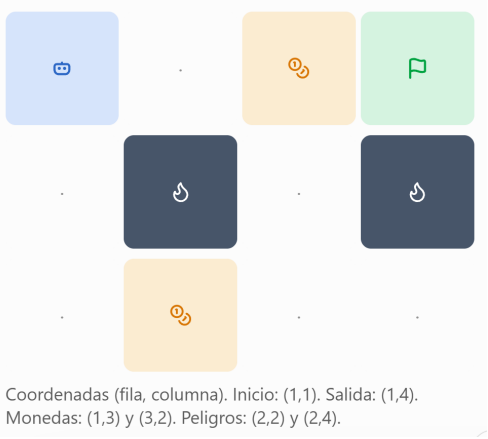




# Ejemplo: profundidad

Base de hechos inicial:

f-2: (visitada 1 1)
f-4: (moneda 1 3)
f-6: (peligro 2 2)
f-7: (peligro 2 4)
f-8: (salida 1 4)
f-9: (conexion 1 1 1 2)
f-10: (conexion 1 1 2 1)
f-11: (conexion 1 2 1 1)
f-12: (conexion 1 2 1 3)
f-13: (conexion 1 3 1 2)
f-14: (conexion 1 3 1 4)
f-15: (conexion 1 3 2 3)
...
f-30: (visitada 2 1)
f-32: (visitada 3 1)
f-34: (visitada 3 2)
f-35: (puntos 1)
f-36: (posicion 3 3)
f-37: (visitada 3 3)




Se asume que se crean primero las activaciones que visitan celdas en la misma fila

| Salience | Activación |
|---|---|
| 0 | mover: hacia (2,3) |
| 0 | mover: hacia (3,4) |

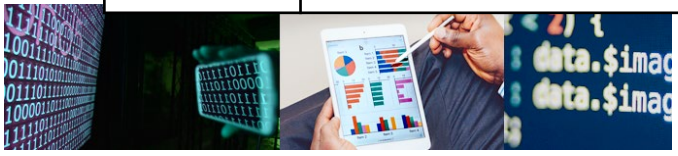
Nueva base de hechos:

...
~~f-36: (posicion 3 3)~~
f-37: (visitada 3 3)
f-38: (posicion 2 3)
f-39: (visitada 2 3)

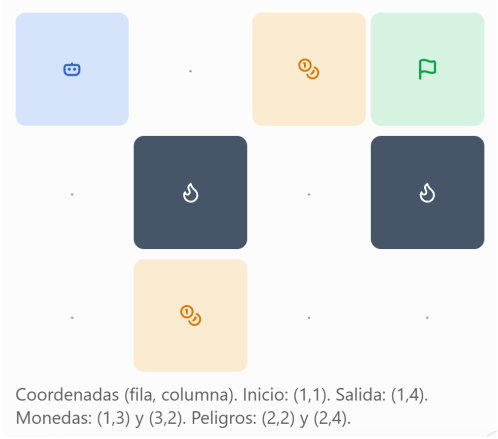
Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 0 | mover: hacia (2,3) |
| 0 | mover: hacia (3,4) |
| 0 | mover: hacia (1,3) |





# Ejemplo: profundidad




Se asume que se crean primero las activaciones que visitan celdas en la misma fila **y en columnas anteriores**

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,3) |
| | |

Nueva base de hechos:

...

~~f-38: (posicion 2 3)~~

f-39: (visitada 2 3)

f-40: (posicion 1 3)

f-41: (visitada 1 3)

Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,3) |
| 50 | recoger-moneda (1,3) |
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (1,2) |

| Salience | Activación |
|---|---|
| 50 | recoger-moneda (1,3) |
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (1,2) |

Nueva base de hechos:

~~f-35: (puntos 1)~~ ~~f-4: (moneda 1 3)~~

...

f-40: (posicion 1 3)

f-41: (visitada 1 3)

f-42: (puntos 2)

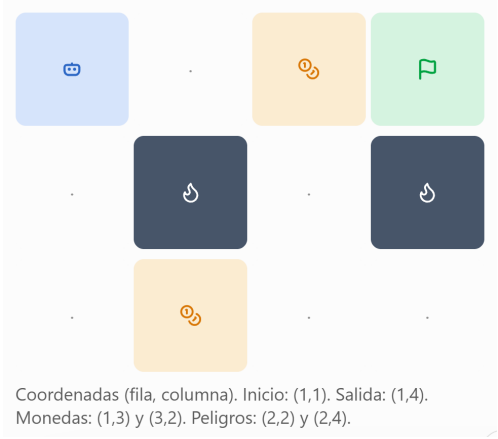
Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 50 | recoger-moneda (1,3) |
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (1,2) |





# Ejemplo: profundidad




Se asume que se crean primero las activaciones que visitan celdas en la misma fila **y en columnas anteriores**

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (1,2) |

Nueva base de hechos:

...

~~f-40: (posicion 1 3)~~

f-41: (visitada 1 3)

f-42: (puntos 2)

f-43: (posicion 1 4)

f-44: (visitada 1 4)

Se actualiza la agenda:

| Salience | Activación |
|---|---|
| 0 | mover: hacia (1,4) |
| 0 | mover: hacia (1,2) |
| 100 | encontrar-salida |

| Salience | Activación |
|---|---|
| 100 | encontrar-salida |

**Finaliza la ejecución**





# Conclusiones: estrategias de resolución de conflictos

Las estrategias de control determinan cómo se seleccionan y ejecutan las reglas aplicables durante el proceso de inferencia.

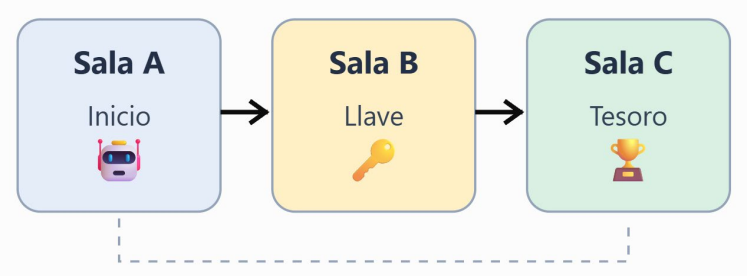
- Prioridades explícitas (salience): permiten establecer qué reglas tienen preferencia sobre otras. Las activaciones con mayor prioridad se ejecutan primero.
- Estrategia de profundidad (depth): entre activaciones con la misma prioridad, favorece las más recientes.
- Estrategia de anchura (breadth): entre activaciones con la misma prioridad, favorece las más antiguas.
- Agenda dinámica: al ejecutar una regla, la BH puede modificarse, provocando la aparición de nuevas activaciones o la eliminación de otras.
- Influencia en la solución: distintas estrategias pueden producir diferentes secuencias de ejecución y resultados, aunque se utilicen las mismas reglas y hechos iniciales.
- Limitaciones: las estrategias de control no garantizan por sí solas encontrar una solución ni obtener la mejor. Es necesario diseñar adecuadamente la representación del problema y las reglas.





# Ejercicio

Un explorador se encuentra en una mazmorra con tres salas:




El explorador comienza en la sala A. Puede recoger una llave en la sala B y abrir la puerta del tesoro en la sala C.

```
Base de hechos inicial
(deffacts inicio
   (posicion A)
   (camino A B)
   (camino A C)
   (camino B C)
   (llave B)
   (puerta C cerrada)
   (energia 2)
)
```

```
; R1: Moverse a otra sala
(defrule mover
   ?p <- (posicion ?x)
   (camino ?x ?y)
   ?e <- (energia ?n)
   (test (> ?n 0))
=>
   (retract ?p ?e)
   (assert (posicion ?y))
   (assert (energia (- ?n 1)))
)

; R2: Recoger la llave
(defrule recoger-llave
   (posicion ?x)
   ?l <- (llave ?x)
=>
   (retract ?l)
   (assert (tiene-llave))
)
```

```
; R3: Abrir la puerta
(defrule abrir-puerta
   (posicion ?x)
   (tiene-llave)
   ?p <- (puerta ?x cerrada)
=>
   (retract ?p)
   (assert (puerta ?x abierta))
)

; R4: Conseguir el tesoro
(defrule conseguir-tesoro
   (posicion C)
   (puerta C abierta)
=>
   (printout t "¡Tesoro
                conseguido!" crlf)
   (halt)
)
```





# Ejercicio

Se supone que las activaciones se generan en orden R4 → R3 → R2 → R1 y, dentro de R1, mover(A,B) antes que mover(A,C). Todas tienen prioridad 0.
Se pide:

1. Ejecuta el sistema utilizando la estrategia de anchura. Para cada ciclo, indica la agenda, la regla seleccionada y los cambios producidos en la BH.
2. Repite el ejercicio utilizando la estrategia de profundidad. ¿Se ejecutan las mismas reglas en el mismo orden? ¿Se consigue el tesoro en ambos casos?
3. Prioridades explícitas. Modifica las reglas para establecer las siguientes prioridades, repite la ejecución y explica qué cambia respecto a los apartados anteriores:

| Regla | Salience |
|---|---|
| conseguir-tesoro | 100 |
| abrir-puerta | 50 |
| recoger-llave | 20 |
| mover | 0 |

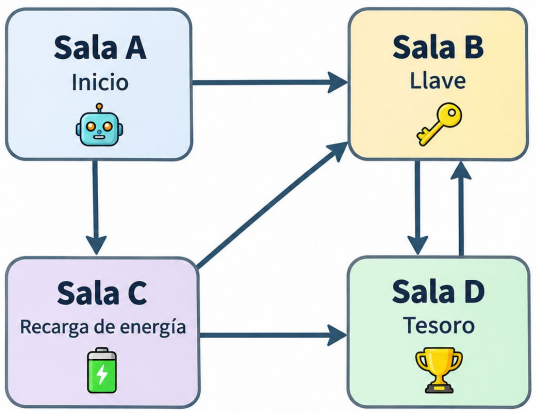
4. Responde razonadamente: ¿puede el explorador quedarse sin energía antes de conseguir el tesoro? ¿Garantiza alguna de las estrategias encontrar una solución? ¿Qué cambios introducirías en las reglas para evitar movimientos que conducen a un fracaso?





# Ejercicio

Supongamos ahora que se dispone de este mapa:




Se pide:

1. Añade una regla R5 que permita cargar el nivel de energía del explorador hasta 10 unidades.
2. ¿La regla R4 sigue siendo válida?
3. ¿Es necesario modificar la regla R1 para permitir los nuevos movimientos entre salas?
4. Ejecuta el sistema utilizando las estrategias de anchura y profundidad, suponiendo que las activaciones se generan en orden R4 → R3 → R2 → R1 → R5 y que las instancias de mover se generan en orden alfabético de la sala destino.

Base de hechos inicial:

```
(deffacts inicio
   (posicion A)
   (camino A B)
   (camino A C)
   (camino B D)
   (camino C D)
   (camino C B)
   (camino D B)
   (llave B)
   (recarga C)
   (puerta D cerrada)
   (tesoro D)
   (energia 2)
)
```



